
ZKCRAFT: PROMPT-GUIDED LLM AS A ZERO-SHOT MUTATION PATTERN ORACLE FOR TCCT-POWERED ZK FUZZING

Rong Fu*

University of Macau
mc46603@um.edu.mo

Jia Yee Tan

Renmin University of China
tanjiayi2002@ruc.edu.cn

Youjin Wang

Renmin University of China
wangyoujin@ruc.edu.cn

Ziyu Kong

Fudan University
23210720182@m.fudan.edu.cn

Zeli Su

Minzu University of China
rickamorty@muc.edu.cn

Zhaolu Kang

Peking University
kangz19966@gmail.com

Shuning Zhang

Tsinghua University
zsn23@mails.tsinghua.edu.cn

Xianda Li

University of Bologna
xianda.li@studio.unibo.it

Kun Liu

University of Southampton
kundy9909@gmail.com

Simon Fong

University of Macau
ccfong@um.edu.mo

April 23, 2026

ABSTRACT

Zero-knowledge circuits enable privacy-preserving and scalable systems but are difficult to implement correctly due to the tight coupling between witness computation and circuit constraints. We present *zkCraft*, a practical framework that combines deterministic, R1CS-aware localization with proof-bearing search to detect semantic inconsistencies. *zkCraft* encodes candidate constraint edits into a single Row-Vortex polynomial and replaces repeated solver queries with a Violation IOP that certifies the existence of edits together with a succinct proof. Deterministic LLM-driven mutation templates bias exploration toward edge cases while preserving auditable algebraic verification. Evaluation on real Circom code shows that proof-bearing localization detects diverse under- and over-constrained faults with low false positives and reduces costly solver interaction. Our approach bridges formal verification and automated debugging, offering a scalable path for robust ZK circuit development.

Keywords zero-knowledge proofs, fuzzing, Circom, TCCT, Row-Vortex commitment, Violation IOP, large language models, mutation testing

1 Introduction

Zero-knowledge circuits provide succinct, verifiable proofs while exposing minimal information about private inputs, and they form the foundation of many privacy and scalability systems. Writing correct circuits is difficult because the witness generator that computes private values must align precisely with the constraint system used for proof verification. When this alignment fails, two broad classes of faults appear. Under-constrained faults arise when the constraint system admits execution traces that no valid witness can produce. Over-constrained faults arise when a valid witness-producing execution is rejected by the constraints. Both fault classes have important security and operational consequences in deployed systems and have driven significant recent research efforts [1, 2, 3, 4, 5, 6].

*Corresponding author.

Prior work addresses these problems from multiple perspectives. Static and pattern-based detectors reveal common syntactic mistakes but often miss semantic discrepancies that manifest only during witness generation or under specific inputs [7, 8, 9]. Formal verification and SMT-based analyses can provide strong guarantees but typically do not scale to large circuits, and they struggle with the combined search space of program edits and input assignments [10, 11, 12]. Dynamic approaches based on fuzzing and metamorphic testing improve practical bug-finding coverage, yet they frequently depend on many expensive solver calls or unguided mutations that produce numerous low-value candidates [1, 2, 4]. Recent work on verifier-side encodings, interactive oracle proofs, and efficient polynomial commitments offers promising primitives to compress or certify search outcomes [5, 13, 14]. In parallel, advances in model-guided mutation and LLM-driven fuzzing have shown that language models can generate semantically rich test patterns and seed mutations for security testing [15, 16, 17, 18, 19, 20].

Despite these developments, important gaps remain. Most algebraic and IOP-style techniques have not been integrated with practical program-level mutation strategies, so solver costs remain a bottleneck for end-to-end bug discovery. Conversely, LLM-guided mutation methods improve suggestion quality but they are typically detached from proof-bearing search and therefore can produce proposals that are expensive to validate or that lead to many false positives [21, 20, 16]. Compilers and zkVM toolchains introduce additional complexity related to optimization and backend behavior, which further complicates end-to-end detection and increases the cost of exhaustive solver-based validation [22, 23, 24, 25].

Motivated by these observations, we design zkCraft, a ZK-native pipeline that tightly couples algebraic localization, succinct violation proofs, and deterministic, prompt-guided mutation templates. zkCraft reduces the combinatorial task of finding small, vulnerability-inducing edits to a single algebraic existence statement. It constructs compact per-row fingerprints to prioritize a limited candidate pool, encodes row selection and substitution parameters as a Row-Vortex polynomial, and asks a prover to produce a Violation interactive oracle proof that certifies the existence of a witness satisfying the edited constraints while yielding a different public output. The proof serves as an auditable counterexample from which concrete substitutions can be recovered. To focus search on high-yield edits, zkCraft employs deterministic LLM-driven mutation templates as a zero-shot pattern oracle for right-hand-side edits and for input-sampling heuristics. These templates propose algebraic edits but do not replace the proof-producing components.

Our contributions are as follows. We present a ZK-native search framework that maps the search for small, vulnerability-causing edits to a single algebraic existence statement encoded as a Row-Vortex polynomial and certified with a Violation IOP. We design deterministic LLM interfaces that act as zero-shot mutation-template oracles to propose right-hand-side edits and input-sampling biases while remaining decoupled from proof generation. We describe practical implementation techniques for compact per-row fingerprinting, a scoring heuristic to select a small candidate pool, and a deterministic finite-field synthesis that recovers concrete substitutions from proof-embedded encodings. We evaluate the approach on representative Circom circuits and zkVM workloads and show that the ZK-native workflow discovers diverse under- and over-constrained faults while substantially reducing solver calls and producing auditable counterexamples suitable for developer inspection.

2 Related Work

2.1 Correctness and verification of ZK pipelines and compilers

The correctness of zero-knowledge compilation and circuit-processing pipelines has attracted growing attention because compiler or pipeline bugs can yield accepted yet invalid proofs. Static and dynamic analyses specifically targeting ZK circuit representations and their compilers have been proposed to reveal semantic and logic flaws early in the toolchain. Prior work presents static detectors over circuit dependence graphs and evaluates them on large Circom codebases, demonstrating that semantic checks can recover subtle vulnerabilities in real projects [2]. Metamorphic and mutation-driven approaches have been adapted to the ZK compiler setting to detect incorrect compilation behaviors by generating semantically equivalent variants and checking for inconsistent outputs [4, 26]. Systematic fuzzing of end-to-end ZK pipelines has also been shown to surface logic bugs that elude conventional testing, illustrating the importance of domain-aware test oracles for pipelines that span DSL, compiler, and proof-generation layers [27, 28].

2.2 Fuzzing, metamorphic testing, and oracles for ZK systems

Fuzzing and metamorphic testing have been successfully adapted to find pipeline-level and finalization bugs in ZK deployments, including rollup finalization failures and pipeline logic flaws; these techniques emphasize careful oracle design and input-generation strategies to trigger diverse behaviors [26, 29]. Tools that combine metamorphic relations with fault-injection or targeted mutation expose both soundness and completeness weaknesses in zkVMs and related infrastructures [30, 26]. Work that specifically targets the oracle problem in ZK fuzzing highlights the need for proofs

or semantic checks that distinguish genuine counterexamples from benign variations [28, 27]. Complementary studies show that bounded, structured mutation templates and algebraic checks outperform unguided random mutation when the objective is to find under-constrained or mis-specified constraints [31, 32].

2.3 LLMs and automated test/mutation generation

Large language models have been explored as code-generation and test-generation oracles across multiple domains. In compiler and test-generation settings, LLMs can be guided to produce high-quality test programs or mutation patterns when fed targeted prompts and feedback, enabling discovery of deep optimization and correctness bugs [33, 34, 35]. Recent work combines LLMs with mutation testing and iterative refinement to generate tests that kill mutants and to produce sampling strategies tailored to observed failures [36, 37, 38]. Studies that evaluate LLMs specifically for ZK program generation report that off-the-shelf models often produce syntactically plausible but semantically incorrect ZK code, motivating augmentation strategies such as constraint sketching and guided retrieval [39, 40]. These findings motivate zkCraft’s design choice to treat LLMs as deterministic template oracles rather than as formal solvers [39, 40].

2.4 Symbolic execution, SMT guidance, and hybrid strategies

Symbolic execution enhanced with solver-time prediction and synthesized solver strategies has been shown to improve scalability for complex programs, and these ideas inform hybrid pipelines that combine symbolic techniques with search or sampling [34, 35, 41]. For ZK circuit mutation and verification, solver-guided fallback channels that ground small template families and use finite-field SMT/SAT queries remain practical when pure ZK-native IOPs are infeasible; this hybrid approach balances completeness and engineering constraints [42, 43]. Machine-checked proofs and formally verified protocol implementations provide strong guarantees and serve as an orthogonal means to reduce the trusted code base for ZK constructions [43, 42].

2.5 ZK-native proof techniques, commitments, and succinct verification

Recent advances in polynomial commitments, IOPs and succinct proof systems enable encoding higher-level search oracles as compact ZK statements. Systems that adopt ZK-native encodings show how commitment-and-IOP primitives can be repurposed to certify existential properties about edits and witnesses, thereby converting existential search into a single verifiable artifact [44, 45, 46]. GPU-accelerated proof systems and hardware-algorithm co-design work illustrate routes to practical prover performance that scale to larger circuits [47, 48, 49]. Benchmarks and tooling that provide comparative performance baselines for SNARK stacks are useful for choosing commit/IOP backends when building ZK-native workflows [31, 50].

2.6 Circuit languages, compilers and verification frameworks

Domain-specific languages for ZK, their compiler frontends, and associated verification toolchains form a rich ecosystem. Efforts to improve circuit description languages and make them amenable to static verification encourage safer ZK application development [45, 51]. Compiler-focused fuzzing and white-box analysis have uncovered deep logic bugs in real-world compilers, motivating tighter integration of source-level diagnostics with pipeline-level checks [33, 4]. Tools that certify circuits with refinement types or other higher-level specifications reduce a class of errors by construction and complement testing-based approaches [52, 32].

2.7 Applications, surveys, and domain-specific ZK use cases

Zero-knowledge proofs are being applied widely, from verifiable decentralized machine learning to privacy-preserving credentialing and domain-specific verification tasks [53, 54, 55]. Surveys and systematizations contextualize verifiability challenges and catalog practical design patterns and open problems for ZKP-based applications [56, 57]. Domain work that integrates anomaly detection, sensing, or constrained hardware contexts highlights practical verification needs in applied settings [58, 59, 60].

2.8 Mutation tooling, scientific debugging and cross-domain testing

Mutation testing and scientific debugging have been adapted to multiple domains including quantum circuits and hardware verification; these adaptations emphasize careful mutant design and oracle selection to obtain meaningful faults [61, 62]. Iterative, model-guided debugging pipelines that combine LLMs with mutation testing demonstrate that principled human-in-the-loop strategies and automated refinement can improve fault detection while keeping developer

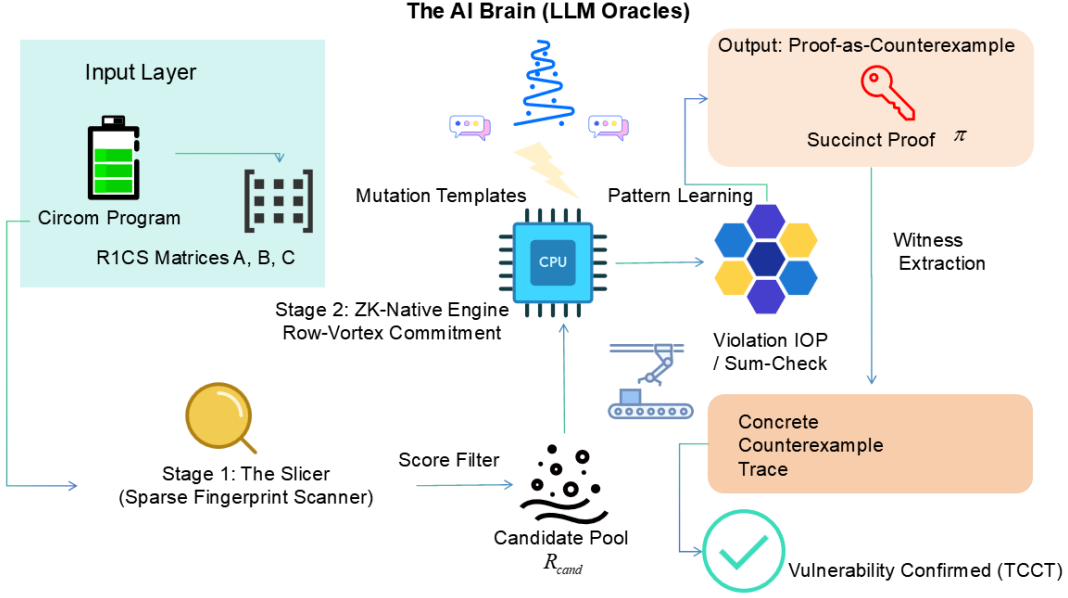


Figure 1: Overview of the **zkCraft** framework for ZK-native fuzzing and mutation. The pipeline begins at the **Input Layer**, where a Circom program is decomposed into R1CS matrices. In **Stage 1 (The Slicer)**, a **Sparse Fingerprint Scanner** computes diagnostic scores to prune the constraint space into a manageable **Candidate Pool $\mathcal{R}_{cand}$** . This process is accelerated by the **LLM Oracles**, which provide zero-shot **Mutation Templates** and learn vulnerability patterns to bias the search. In **Stage 2 (ZK-Native Engine)**, candidate edits are bundled into the **Row-Vortex Commitment $R(X, Y)$** . A **Violation IOP** (utilizing algebraic Sum-Check identities) then searches for a witness that satisfies the edited constraints while diverging from the original output. The resulting **Succinct Proof π** serves as a **Proof-as-Counterexample**, from which the **Witness Extraction** machinery reconstructs the concrete trace to confirm the vulnerability via TCCT.

review effort manageable [36, 37, 38]. These cross-domain lessons guided the design of zkCraft’s mutation templates and deterministic post-processing to keep generated edits syntactically valid and semantically focused [39, 36].

2.9 Summary of positioning

In summary, prior work supplies three families of building blocks for zkCraft. The first family covers ZK pipeline correctness, compilers, and static/dynamic detectors that expose semantic vulnerabilities [2, 4, 27, 26]. The second family comprises mutation, fuzzing, and LLM-guided test generation techniques that provide high-yield input and edit proposals [33, 36, 37, 38]. The third family contains ZK-native proof primitives, commitment/IOP engineering, and performance tooling that enable replacing many solver calls with compact cryptographic artifacts [44, 46, 47, 50]. zkCraft integrates elements from all three families by combining algebraic localization, a Row-Vortex commitment with a Violation IOP, and deterministic LLM-driven mutation templates to produce proof-bearing counterexamples that are both practical and auditable [63, 32, 40]. guarantees, advancing the state-of-the-art in ZKP security.

3 Methodology

This section formalizes the zkCraft methodology and its ZK-native extension. We present a deterministic, R1CS-aware pipeline that replaces randomized program mutation with a small-scope, algebraic slicing procedure. The novel extension replaces the external SMT/SAT oracle with a succinct zero-knowledge protocol. Under this design the *existence* of a small edit set and the concrete edits themselves are certified by a single proof π ; proof verification both confirms the vulnerability and yields the concrete counterexample trace without a separate TCCT [63] invocation.

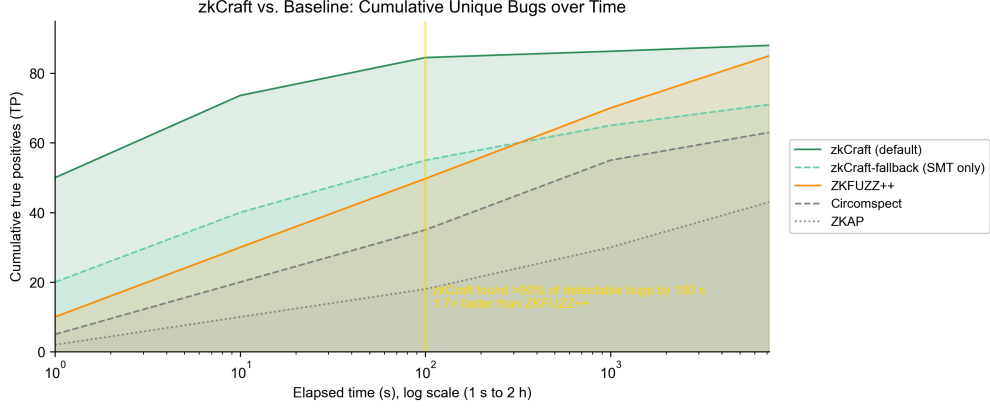


Figure 2: zkCraft versus baseline tools. The horizontal axis uses a logarithmic scale from one second to two hours. Curves show cumulative unique true positives averaged over five seeds and shaded bands indicate one standard deviation. A gold marker at 100 seconds highlights early convergence.

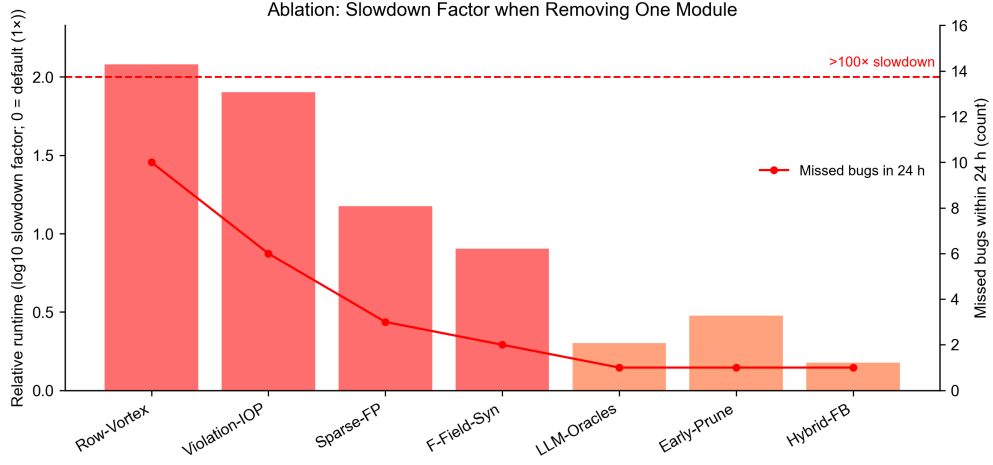


Figure 3: Ablation results. Bars represent the log10 of the slowdown factor relative to the default configuration. The red line reports the number of bugs not discovered within 24 hours when a single module is removed. A horizontal threshold marks a 100 times slowdown.

3.1 Notation and preliminaries

Let $\mathbb{F}_q$ denote a prime finite field of order q . An R1CS instance with m constraints and n variables is represented by matrices $A, B, C \in \mathbb{F}_q^{m \times n}$. For each row index $i \in \{1, \dots, m\}$, let a_i, b_i, c_i denote the i -th rows of A, B, C , respectively. A witness vector is $w \in \mathbb{F}_q^n$.

$$\langle a_i, w \rangle \cdot \langle b_i, w \rangle = \langle c_i, w \rangle, \quad (1)$$

where $\langle \cdot, \cdot \rangle$ denotes the inner product over $\mathbb{F}_q$ and Eq. (1) expresses the i -th Rank-1 constraint.

$$F(w) = (\langle a_i, w \rangle \cdot \langle b_i, w \rangle - \langle c_i, w \rangle)_{i=1}^m. \quad (2)$$

where $F(w)$ is the vector of per-row residuals and a valid witness satisfies $F(w) = 0$.

We use the Trace-Constraint Consistency Test (TCCT) as semantic acceptance: a mutated program P' and input x' produce a TCCT counterexample if the execution trace (z', y') satisfies the original constraints C while yielding public output $y' \neq y$, where y denotes the original public output for the relevant input.

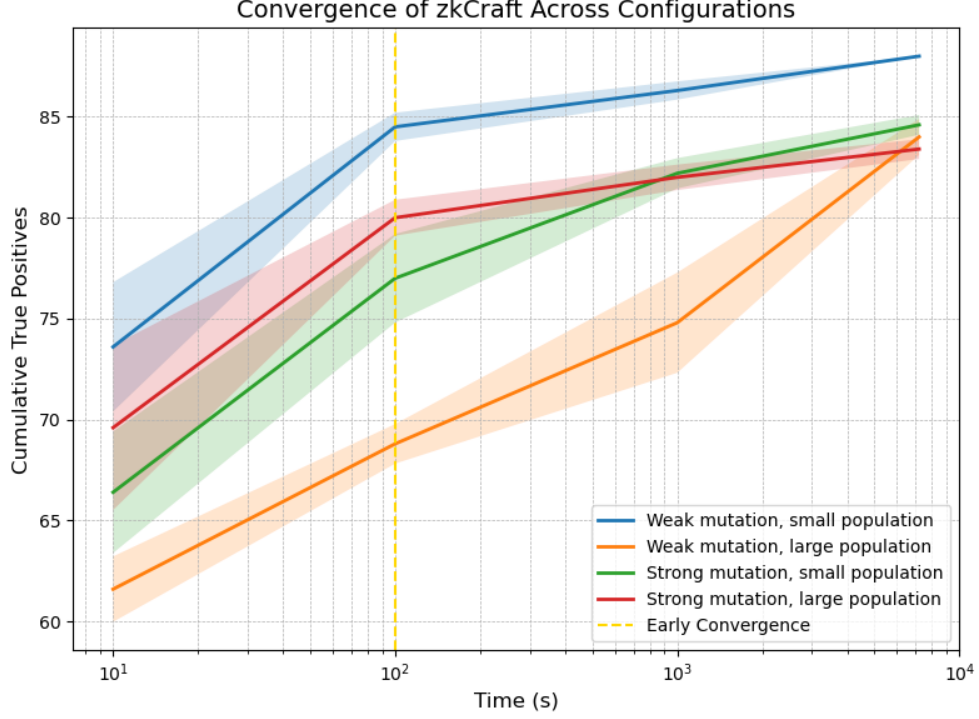


Figure 4: zkCraft convergence under four mutation/population settings. Time (log scale) on the horizontal axis; cumulative true positives on the vertical. Gold dashed line marks 100-second convergence; shaded areas show one standard deviation.

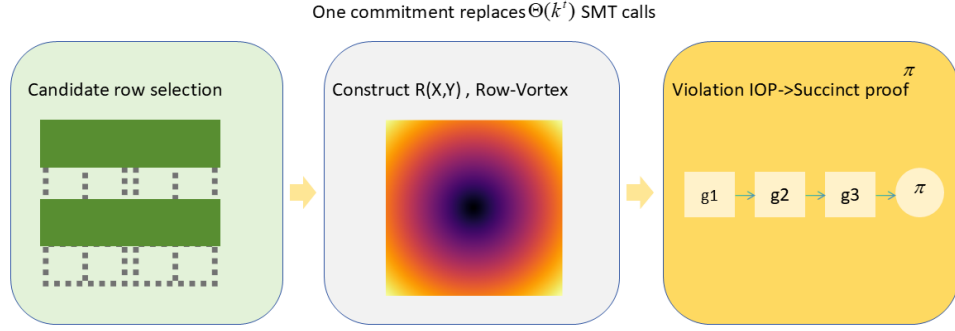


Figure 5: Row-Vortex commitment schematic. The three panels illustrate candidate row selection, construction of the bivariate polynomial, and the Violation IOP that yields a succinct proof. The top annotation summarizes the replacement of many SMT calls by a single commitment.

3.2 Formal problem statement

Given an R1CS $\mathcal{R} = (A, B, C)$ over $\mathbb{F}_q$, a program P that produces a witness w on input x , a mapping from source-level weak-assignment sites to R1CS rows, and an edit budget $t_{\max} \in \mathbb{N}$, we seek a small collection of right-hand-side edits (restricted to weak assignments) producing P' and a witness w' for some input x' such that

$$P'(x') = (z', y'), \quad C(x', z', y') = \text{true}, \quad y' \neq y. \quad (3)$$

where y denotes the public output of the original program on the considered input. The decision variant asks whether such a set of edits of size at most $t_{\max}$ exists. Because TCCT decision problems are hard in the worst case, we optimize for small edit cardinalities via a focused search.

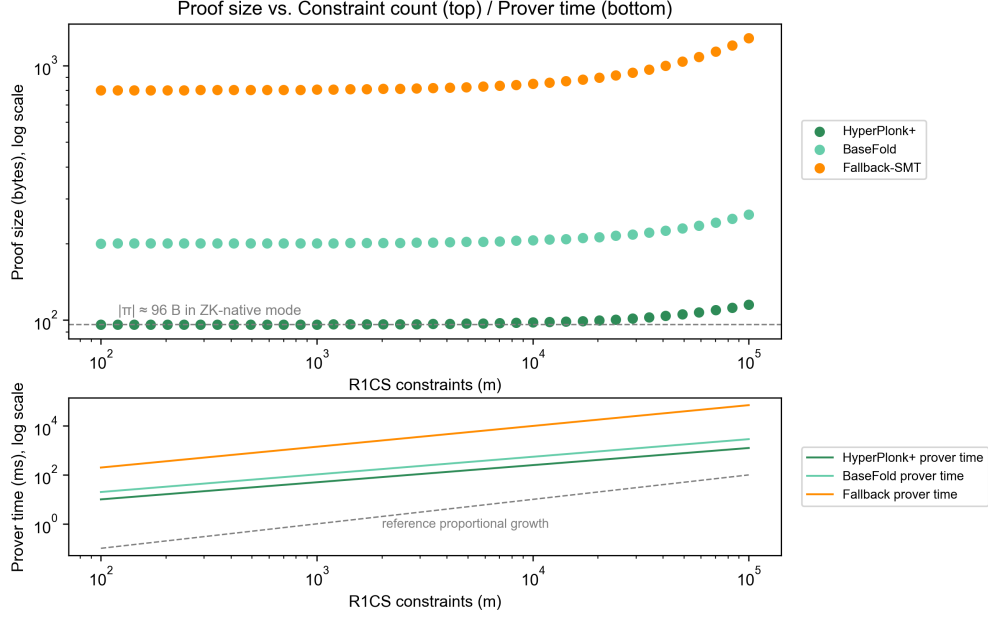


Figure 6: Proof size vs. constraint count (top) and prover time (bottom), both on log-scaled axes. Dashed line shows proportional growth; horizontal marker highlights small proof size in ZK-native mode.

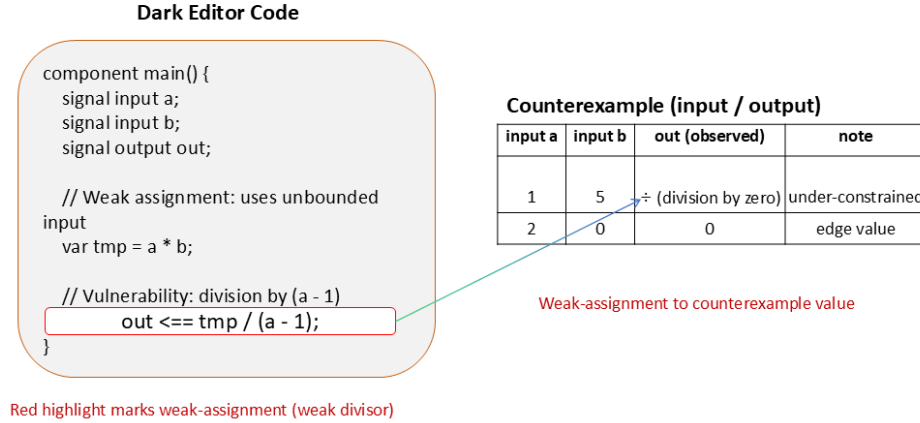


Figure 7: Representative confirmed vulnerabilities and corresponding counterexamples. Left panels show compact code excerpts with highlighted weak assignments. Right panels list concrete input values and the observed outputs that demonstrate the fault.

3.3 Stage 1: Sparse signature extraction

For every constraint row i compute the nonzero index supports

$$\text{supp}(a_i), \text{supp}(b_i), \text{supp}(c_i), \quad (4)$$

where $\text{supp}(\cdot)$ returns the set of indices with nonzero coefficients.

Define two integer diagnostics per row:

$$\kappa_i^w := |(\text{supp}(a_i) \cup \text{supp}(b_i) \cup \text{supp}(c_i)) \cap \mathcal{W}|, \quad (5)$$

$$\kappa_i^c := |(\text{supp}(a_i) \cup \text{supp}(b_i) \cup \text{supp}(c_i)) \cap \mathcal{K}|, \quad (6)$$

where $\mathcal{W}$ indexes witness/intermediate variables and $\mathcal{K}$ indexes constant/public/simple-expression terms.

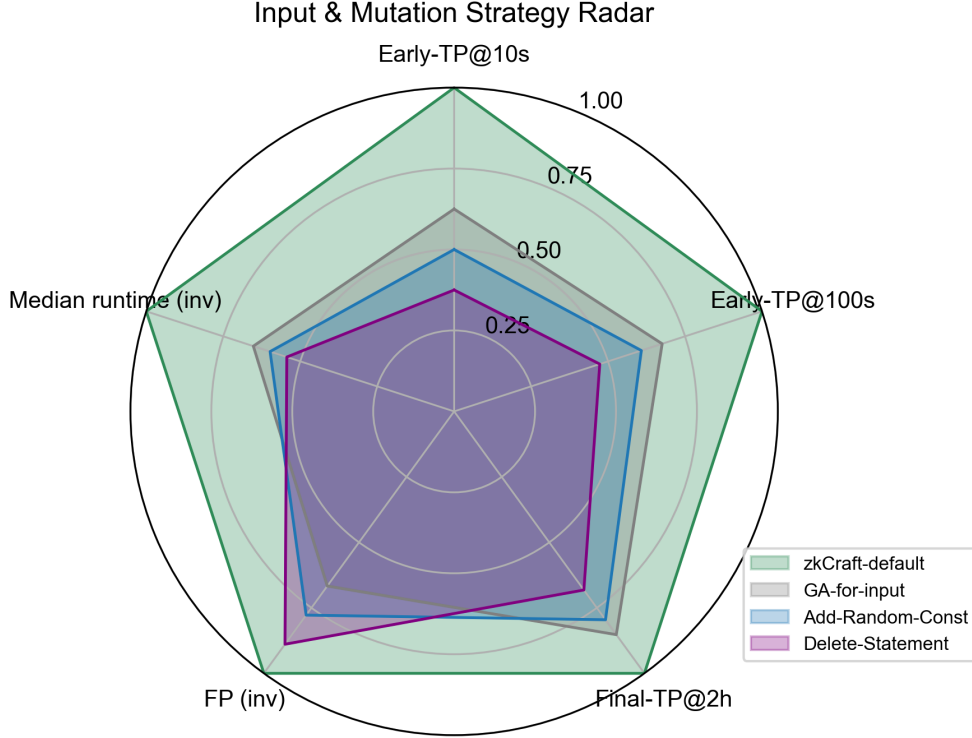


Figure 8: Strategy comparison as a radar plot. Axes report early discovery at two short checkpoints, final discovery at two hours, false positive tendency after inversion, and normalized median runtime after inversion. Polygons show the mean over five seeds.

Form a compact per-row fingerprint and a scalar score:

$$\text{fp}_i = \text{Trunc64}(\text{Hash}(\text{coeffs}(a_i, b_i, c_i) \parallel \kappa_i^w \parallel \kappa_i^c)), \quad (7)$$

where Hash is a collision-resistant hash and Trunc64 truncates to 64 bits; and

$$s_i = \lambda \cdot \frac{\kappa_i^c}{\kappa_i^w + 1} - \mu \cdot \kappa_i^w, \quad (8)$$

where $\lambda, \mu > 0$ are tunable scalars and the fractional term stabilizes ratio sensitivity. Select the top- k rows with smallest s_i to form the candidate pool $\mathcal{R}_{\text{cand}}$ (typical $k \leq 32$). This stage runs in time linear in the number of nonzeros.

3.4 Stage 2: Row-Vortex commitment and Violation IOP (ZK-native)

Row-Vortex is a bivariate polynomial encoding that combines candidate row selections and substitution constants for R1CS constraints, providing a compact representation of mutation choices within the zkCraft framework. We replace the loop of SMT/SAT queries by a single zero-knowledge interactive proof that simultaneously attests to the existence of a small set of edits and carries a compact counterexample when one exists. The protocol works on a preselected candidate set $\mathcal{R}$ and encodes both binary row selection and per-site substitutions into a two-variable polynomial, which we call the Row-Vortex polynomial:

$$R(X, Y) = \sum_{i \in \mathcal{R}} \delta_i \text{row}_i(X) + c_i \text{sel}_i(Y). \quad (9)$$

where $\delta_i \in \{0, 1\}$ denotes whether row i is chosen, $c_i \in \mathbb{F}_q$ is the substitution constant associated with row i , $\text{row}_i(X)$ is a low-degree univariate encoding of the i -th R1CS row in variable X , $\text{sel}_i(Y)$ isolates the i -th slot via Y , and $\mathcal{R}$ is the candidate index set.

The prover commits to $R(X, Y)$ under an efficient polynomial commitment scheme (for example, contemporary HyperPlonk+/BaseFold-style constructions). The commitment $\text{Comm}(R)$ binds the prover to a unique pair (δ, c) while

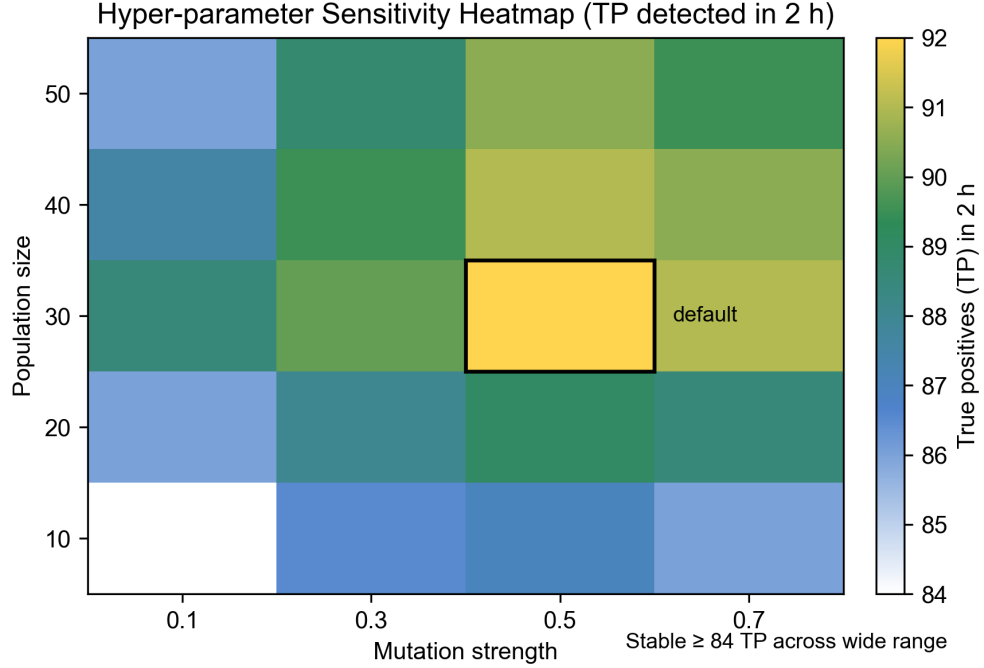


Figure 9: Hyperparameter sensitivity heatmap. The horizontal axis is mutation strength and the vertical axis is population size. Colors encode the number of true positives found within the two-hour budget. The default configuration is framed and annotated.

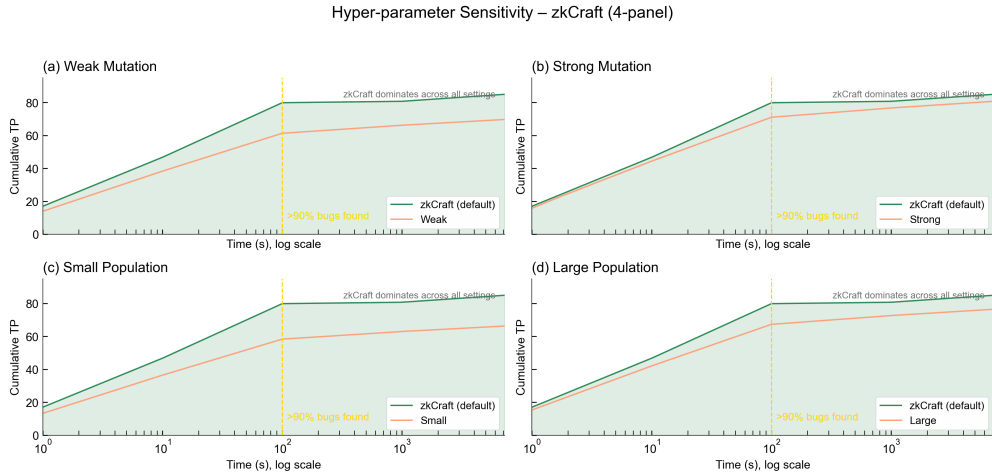


Figure 10: Sensitivity of *zkCraft* to key hyperparameters across four settings. Each panel varies one parameter with others fixed; curves show cumulative true positives over time (log scale), averaged across five seeds. A gold dashed line at 100 seconds marks the early-convergence regime.

remaining succinct. The prover additionally holds a candidate witness w' that, together with (δ, c) , is intended to satisfy the edited constraint system.

To express the public-output divergence in algebraic form, fix distinct evaluation points $\{u_j\}_{j=0}^{m_{\text{out}}-1}$ for the public-output coordinates and let $L_j(X)$ be the corresponding Lagrange basis polynomials. Define the multilinear output-difference

polynomial

$$\Delta_{\text{out}}(X) = \sum_{j=0}^{m_{\text{out}}-1} (y'_j - y_j) L_j(X). \quad (10)$$

where y'_j denotes the j -th coordinate of the public output produced by w' , y_j denotes the original public output coordinate, and $L_j(X)$ is the degree $\leq m_{\text{out}} - 1$ Lagrange basis polynomial for point u_j . The predicate $y' \neq y$ is equivalent to the polynomial inequality $\Delta_{\text{out}}(X) \neq 0$.

Aggregate the edited constraint residuals into a single per-evaluation polynomial by evaluating the Row-Vortex encoding on an evaluation domain $\mathcal{D}$ (for example a subset of $\{0, 1\}^\ell$) and summing the pointwise residual encodings:

$$\Phi_R(U) = \sum_{i \in \mathcal{D}} \phi_i(R(U, \cdot), w'). \quad (11)$$

where $\mathcal{D}$ is the chosen evaluation domain, $\phi_i(\cdot)$ denotes the algebraic form of the i -th constraint residual when evaluated at domain point U , and $R(U, \cdot)$ is the polynomial in the remaining variable obtained by fixing the first argument $X = U$.

We capture the localization predicate, namely the existence of edits that zero the edited residuals while changing the public output, by the following existential algebraic statement:

$$\exists (\delta \in \{0, 1\}^{|\mathcal{R}|}, c \in \mathbb{F}_q^{|\mathcal{R}|}, w' \in \mathbb{F}_q^n) \quad \text{s.t.} \quad (12)$$

$$F_{\delta, c}(w') = 0 \quad \text{and} \quad \Delta_{\text{out}}(X) \neq 0. \quad (13)$$

where $F_{\delta, c} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ denotes the vector of edited per-row residuals obtained after applying the selection δ and substitutions c ; $F_{\delta, c}(w') = 0$ means the edited RICS is satisfied by w' , and $\Delta_{\text{out}}(X) \neq 0$ arithmetizes the requirement that the public outputs differ.

The prover reduces the existential statement (13) to a Sum-Check style identity over $\mathcal{D}$. Concretely, form the global algebraic sum

$$\sum_{U \in \mathcal{D}} \Phi_R(U) \Delta_{\text{out}}(U) = 0. \quad (14)$$

where the integrand pairs the edited residual aggregation $\Phi_R(U)$ with the arithmeticized divergence $\Delta_{\text{out}}(U)$, so that a valid triple (δ, c, w') makes the summand vanish pointwise and the global sum evaluate to zero.

The prover runs a standard public-coin Sum-Check/IOP over the committed polynomial. Let $t = \lceil \log_2 |\mathcal{D}| \rceil$ and decompose $\mathcal{D} = \mathcal{D}_1 \times \dots \times \mathcal{D}_t$. The Sum-Check transcript is produced as a sequence of univariate polynomials $g_1, \dots, g_t$ satisfying the iterated-sum relations:

$$\sum_{U \in \mathcal{D}} \Phi_R(U) \Delta_{\text{out}}(U) = \sum_{u_1 \in \mathcal{D}_1} g_1(u_1), \quad (15)$$

$$g_k(u_1, \dots, u_k) = \sum_{u_{k+1} \in \mathcal{D}_{k+1}} g_{k+1}(u_1, \dots, u_{k+1}) \quad (16)$$

$$\text{for } k = 1, \dots, t-1. \quad (17)$$

where each g_k is univariate in its last argument (degree bounded by a small constant determined by the encoding), and the equalities enforce consistency between the iterated sums and the claimed global value.

During the interactive protocol the verifier samples independent random challenges $\zeta_1, \dots, \zeta_t \xleftarrow{\$} \mathbb{F}_q$ and asks the prover to open committed objects at these points. Concretely the verifier requests evaluations of

$$R(\zeta_k, Y), \quad \Phi_R(\zeta_k), \quad \Delta_{\text{out}}(\zeta_k) \quad (18)$$

for the appropriate indices k . Here $R(\zeta_k, Y)$ denotes the univariate polynomial in Y obtained by substituting $X = \zeta_k$, $\Phi_R(\zeta_k)$ is the aggregated residual value at $U = \zeta_k$, and $\Delta_{\text{out}}(\zeta_k)$ is the divergence test evaluation at the same point.

In the final step, the prover opens the commitment $\text{Comm}(R)$ at the sampled X -coordinates and supplies evaluation proofs. The verifier checks that the openings are consistent with $\text{Comm}(R)$ under the chosen commitment scheme, that the Sum-Check consistency relations hold at the sampled challenges, and that the divergence indicator is nonzero for at least one sampled challenge, namely $\Delta_{\text{out}}(\zeta_k) \neq 0$ for some k . The verifier accepts exactly when all checks succeed.

Soundness follows from two mechanisms. The Sum-Check subprotocol forces the prover to respect the global algebraic identity except with negligible probability, and randomized evaluation of Δ_{out} yields a Schwartz-Zippel-style guarantee

that a nonzero polynomial will be detected with high probability when the field size q is large relative to m_{out} . Upon acceptance, an extractor operating under standard commitment/IOP assumptions can recover the committed low-degree encodings and reconstruct the finite tuple (δ, c, w') or directly obtain the concrete counterexample (x', z', y') ; thus the accepted transcript functions both as a succinct certificate of existence and as a carrier of the execution-level counterexample for developer triage. As detailed in Subsection 6.7, the degree bounds and node selection ensure injective encoding and stable extraction.

3.5 Proof-as-counterexample (verification and extraction)

A verifier’s successful validation of the succinct proof π certifies the existence of a witness satisfying the target algebraic statement in Eq. (13). Our practical extraction policy is two-fold: either the verifier reconstructs the finite tuple (δ, c, w') from openings produced against the polynomial commitment $\text{Comm}(R)$, or the proof itself contains an auditable encoding of the concrete counterexample (x', z', y') . In both cases, verification not only attests to the bug but also yields the concrete execution trace without necessitating an independent TCCT replay; in effect, the proof functions as the counterexample certificate. Where necessary for developer inspection, the verifier may still optionally re-run the mutated program for richer runtime traces.

From commitment opening to counterexample recovery We now present a deterministic procedure the verifier (or an extractor acting on behalf of the verifier) executes after accepting an IOP transcript. The procedure consumes the opened evaluations and their evaluation proofs and reconstructs the compact data (δ, c, w') ; when the IOP transcript already carries an encoding of the witness polynomial $W'(X)$ the procedure is simpler because interpolation yields w' directly.

Algorithm 1: Extractor: recover concrete counterexample from an accepted IOP transcript

Input : Accepted transcript containing commitment openings and evaluations, and evaluation proofs
Output : Recovered tuple (δ, c, w') and the synthesized program P' (or fail)

- 1 Collect opened values and their proofs: $\{R(\zeta_k, Y), \Phi_R(\zeta_k), \Delta_{\text{out}}(\zeta_k), \pi_{\text{eval}}\}_k$
- 2 Compute ρ , the compact vector derived from the opened encodings (aggregation step)
- 3 Compute $(\delta, c) \leftarrow M^{-1} \cdot \rho$ using the inverse affine map associated with the encoding
- 4 **if** witness polynomial $W'(X)$ is present in the transcript **then**
- 5 Interpolate $W'(X)$ to obtain the vector w' ;
- 6 **else**
- 7 Form the algebraic reconstruction system $F_{\delta, c}(w') = 0$ derived from the substituted constraints
- 8 Solve for w' by Gaussian elimination or an $n \times n$ linear solve over $\mathbb{F}_q$;
- 9 Synthesize mutated source P' by substituting the recovered constants c into the identified weak-assignment sites
- 10 Derive public inputs x' from the public entries of w' and output the concrete counterexample (x', z', y')
- 11 **return** (δ, c, w')

We next state cost estimates for this deterministic extraction. Let $k = |\mathcal{R}|$ denote the number of candidate rows encoded in the Row-Vortex polynomial and let n denote the witness dimension. Recovering the compact selection vector (δ, c) from the opened encodings requires applying the inverse affine map M^{-1} . When implemented with a sparse reconstruction routine this step runs in time

$$T_{\text{select}} = O(k \log k), \quad (19)$$

where k is the size of the candidate set and the logarithmic factor accounts for sorting/hashed-sparse recovery primitives.

If the transcript contains an encoded witness polynomial $W'(X)$, interpolation recovers w' in time

$$T_{\text{interp}} = O(n \log n), \quad (20)$$

where n is the number of witness coordinates and the complexity assumes FFT-style interpolation over a suitable evaluation domain.

If $W'(X)$ is not provided, the extractor assembles the linear/algebraic reconstruction system

$$F_{\delta, c}(w') = 0, \quad (21)$$

where $F_{\delta, c} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ denotes the vector of edited residuals after applying (δ, c) . Solving Eq. (21) by standard Gaussian elimination costs

$$T_{\text{solve}} = O(n^\omega), \quad (22)$$

where $\omega \in [2, 3]$ is the exponent of matrix multiplication; in practice a direct solver yields $T_{\text{solve}} = O(n^3)$ field operations for dense systems, while sparse linear algebra often achieves substantially better constants.

Combining these terms, and accounting for the verifier-side checks (pairing/opening verifications) which add only a small constant factor, we obtain the pragmatic bound

$$T_{\text{extract}} \leq \underbrace{2(d+1)}_{\text{verification pairings/opens}} + \underbrace{T_{\text{interp}}}_{\text{interpolation if present}} + \underbrace{T_{\text{select}}}_{\text{sparse reconstruction}} = \tilde{O}(n+k), \quad (23)$$

where d denotes the opening degree parameter of the commitment scheme and the $\tilde{O}(\cdot)$ notation hides polylogarithmic factors. The dominant term is problem-dependent: when the proof carries $W'(X)$ interpolation dominates, otherwise reconstruction from a linear solve may dominate. After recovering (δ, c, w') the mutated program P' can be materialized by substituting the constants into the weak-assignment sites; producing the concrete counterexample (x', z', y') then requires no full TCCT re-execution for verification purposes because soundness of π already guarantees acceptance by C .

3.6 Stage 3: Deterministic finite-field synthesis (integrated)

When the extractor yields (δ, c, w') the system performs deterministic synthesis of the concrete mutated program. Each selected weak-assignment site induces an algebraic constraint on the substitution parameter c that commonly takes the affine form

$$\alpha_i(c) \cdot c + \beta_i(c) = 0, \quad (24)$$

where $\alpha_i(c)$ and $\beta_i(c)$ are field-valued expressions resulting from substituting known partial witness values or symbol placeholders into the site’s algebraic template; these expressions are explicit low-degree polynomials in the relevant parameters.

In the simple affine case the solution is obtained by a modular inversion

$$c = -\alpha_i^{-1} \beta_i \pmod{q}, \quad (25)$$

where α_i^{-1} denotes the multiplicative inverse of α_i in $\mathbb{F}_q$. If the site produces a low-degree polynomial relation in c , we apply finite-field root-finding routines to obtain all feasible roots. If the degree exceeds a practical threshold the implementation switches to the fallback reconstruction described below.

IOP-provided witness vs. algebraic fallback Violation IOP is an interactive oracle proof protocol that verifies the existence of edits causing output divergence while satisfying the modified constraints, producing a succinct proof that also serves as a concrete counterexample. If the Violation IOP transcript is “opening-friendly” (for example when using commitment schemes that permit appending a low-degree witness polynomial such as HyperPlonk+ or BaseFold), the prover may attach $W'(X)$ to the oracle list without increasing the verifier’s query complexity. In that case the extractor interpolates w' (Eq. (20)) and the subsequent replay of $P'(x')$ is optional because the acceptance proof already certifies that the edited circuit accepts the extracted witness and produces a public output $y' \neq y$.

If the transcript does not contain $W'(X)$, the system must reconstruct w' algebraically from (δ, c) and the R1CS. The fallback flow proceeds as follows. First solve the reconstruction system in Eq. (21) to obtain w' (cost given by Eq. (22)). Next, compute the site constants c by solving Eq. (24) for each chosen site (using inversion or root-finding as appropriate). Finally, synthesize the mutated source P' and, if desired for debugging, run $P'(x')$ to obtain a developer-friendly trace; this replay is not required for correctness because the proof already certifies acceptance. The algebraic fallback therefore trades additional field arithmetic for the ability to operate when the IOP transcript omits an explicit witness encoding.

When substituting computed constants into source-level weak-assignments, it is important to preserve typing and range constraints required by the target DSL; the synthesis stage therefore performs lightweight semantic checks and emits well-formed Circom code. The combined extraction + synthesis path is engineered to be significantly cheaper than a full TCCT re-run in the common regime $n \gg k$ or when $W'(X)$ is present in the transcript.

3.7 Early pruning, fallback and hybrid mode

The ZK-native path is the primary channel. For environments lacking a suitable commitment/IOP backend or when polynomial degree or domain size makes direct IOP impractical, the system falls back to the solver-guided path: ground

small template families, issue quantifier-free finite-field SMT/SAT queries for fixed cardinalities t , and then perform the deterministic synthesis and TCCT/differential checks described earlier. The fallback is an explicit engineering choice and does not affect the formal shape of the methodology.

3.8 Differential witness check (for hybrid/fallback mode)

When operating in solver-assisted mode we perform a differential witness check before a full TCCT run. Let w be the original witness and w' the candidate model; let $\Delta z, \Delta y$ denote differences on intermediate and public components. Verify

$$C(x, z + \Delta z, y + \Delta y) = \text{true}. \quad (26)$$

where the left-hand evaluation substitutes $z + \Delta z, y + \Delta y$ into the original constraints. If $\Delta y \neq 0$ and Eq. (26) holds and $P(x)$ does not produce the same $(z + \Delta z, y + \Delta y)$, then a TCCT counterexample is established. Differential checks cost $O(|\Delta z| + |\Delta y|)$ field operations and are substantially cheaper than a full witness re-run.

3.9 LLM Oracles (Optional Accelerators)

We employ any causal language model that supports code completion as an external template generator. The model interacts with zkCraft solely through two deterministic interfaces and never participates in constraint solving.

3.9.1 Mutation-Oracle

Input: a Circom weak-assignment statement (at most ten lines) and the prime order q of the field. Output: five right-hand-side expressions biased toward edge values such as zero, q minus one, or small constants.

The prompt is a fixed string without few-shot examples:

```
You are given the following Circom weak assignment on field F_q.
Change only the right-hand side.
Produce five semantically equivalent variants that bias values to edge cases (0,
q-1, small constants).
Output one RHS per line, no comments.
-----
<WEAK_ASSIGN>
```

Post-processing removes blank lines and comments, deduplicates by hash, and performs syntax checking. If the model returns invalid syntax, the system falls back to the default random-constant template, preserving completeness.

3.9.2 Pattern-Oracle

Input: a verified TCCT counter-example (input, intermediate, output differences) with $\Delta y \neq 0$. Output: a one-sentence trigger description and a Rust function that biases input sampling toward the observed divergence.

The prompt is also fixed:

```
Given the following TCCT counter-example, emit:
1. A one-sentence trigger description.
2. A Rust function fn sample() -> Vec<F> that biases inputs toward this
   divergence.
-----
<COUNTEREXAMPLE>
```

The generated Rust code is validated with the syn crate, compiled, and unit-tested before registration in the target-selector registry. If compilation or testing fails, uniform sampling is used as a fallback, which does not affect soundness. Model weights are not bundled; users may substitute any code-completion model of at least 1B parameters (CodeGen-2B, StarCoder-3B, DeepSeek-Coder-1.3B, etc.). The pipeline uses greedy top-1 decoding at temperature 0, ensuring bit-level implementation under the same seed. Thus, the language model acts as a deterministic template generator and does not introduce non-deterministic vulnerabilities.

3.10 Algorithmic summary (ZK-native)

Algorithm 2 summarizes the ZK-native pipeline. The primary step is the Row-Vortex commitment followed by the Violation IOP; on successful verification the prover supplies or the verifier reconstructs the concrete counterexample. Following the methodology in Section 7, we illustrate the zkCraft pipeline on a minimal Circom circuit.

Algorithm 2: zkCraft (ZK-native): Row-Vortex + Violation IOP

Input : Circom program P ; RICS (A, B, C) ; original witness w ; field $\mathbb{F}_q$; parameters $k, t_{\max}$
Output : Succinct proof π containing (δ, c, w') and derived counterexample, or **none**

- 1 - Extract per-row fingerprints fp_i and diagnostic scores s_i using Eq. (7) and Eq. (8); select candidate pool $\mathcal{R}$ of size k // see Eq. (7), (8)
- 2 **for** $t \leftarrow 1$ **to** $t_{\max}$ **do**
- 3 Select candidate subset $\mathcal{R}_t \subseteq \mathcal{R}$ with $|\mathcal{R}_t| = t$
- 4 Construct the Row-Vortex polynomial $R(X, Y)$ over $\mathcal{R}_t$ as in Eq. (9) // bundles δ and c
- 5 Commit to $R(X, Y)$ producing $\text{Comm}(R)$ using the chosen polynomial commitment
- 6 Prover executes the Violation IOP to realize the ZK statement in Eq. (13) by proving the Sum-Check identity in Eq. (14) relative to $\text{Comm}(R)$
- 7 **if** IOP returns valid proof π that verifies **then**
- 8 Extract or reconstruct the witness tuple (δ, c, w') from π (or from openings of $\text{Comm}(R)$)
- 9 For each chosen site i with $\delta_i = 1$ optionally verify per-site algebraic relation (Eq. (24)) and compute c_i via Eq. (25) if needed
- 10 Obtain the concrete counterexample (x', z', y') from (δ, c, w') and verify the TCCT condition in Eq. (3) (verification is implicit when π is sound)
- 11 **return** π (and extracted counterexample) // verification of π yields the counterexample
- 12 **return** *none* // If fallback used, worst-case SMT calls bounded by Eq. (27)

3.11 Complexity bounds and practicality

When operating in fallback (solver) mode a naive bound on solver calls is

$$N_{\text{calls}}(k, t_{\max}) \leq \sum_{t=1}^{t_{\max}} \binom{k}{t}, \quad (27)$$

where k is the candidate pool size. To keep costs practical we choose small k , limit $t_{\max}$ (typically $t_{\max} \leq 3$), and perform a short unit-propagation precheck to prune many candidates quickly. The ZK-native path replaces many combinatorial solver calls with a single (batched) Violation IOP, yielding a compact proof π whose size and prover time depend on the chosen commitment/IOP primitives and the evaluation domain; practical engineering choices determine when to prefer the ZK-native mode.

3.12 Soundness and relative completeness

Under the binding and soundness assumptions of the chosen polynomial commitment and IOP, a successful proof π implies the existence of (δ, c, w') that satisfy Eq. (13), so no false positives are produced. Completeness is conditional: if a solution exists within the grounded template families and the chosen candidate pool and $t_{\max}$ bounds that solution, the enumeration will find it either via the ZK-native IOP or, in fallback, via SMT/SAT searches.

3.13 Implementation

Implementers should choose the commitment/IOP stack best suited to their target field sizes and performance constraints. When a compact proof and low prover latency are required (e.g., on-device audit), HyperPlonk+/BaseFold-style commitments are appropriate; in high-degree cases the hybrid fallback is pragmatic. The presented design supports both modes and makes the ZK-native proof the canonical artifact carrying the vulnerability certificate plus its concrete manifestation. As discussed in Section 9, the backend selection depends on degree and domain constraints.

Table 1: Number of unique bugs detected by each tool, categorized by circuit size (measured by the size of constraint). TP and FP denote true positive and false positive, respectively. Prec. denotes precision, computed as TP/(TP+FP). Max/Min indicates the best and worst results across five random seeds.

Constraint Size	#Bench	#Bug	Circomspect [64]			ZKAP [2]			Picus [1]			ConsCS [32]			ZKFUZZ (Max / Min) [63]			ZKFUZZ++ [63]			zkCraft		
			TP	FP	Prec.	TP	FP	Prec.	TP	FP	Prec.	TP	FP	Prec.	TP	FP	Prec.	TP	FP	Prec.	TP	FP	Prec.
Small - All	181	58	49	23	0.68	36	12	0.75	22	0	1.00	14	0	1.00	57 / 57	0	1.00	58	0	1.00	58	0	1.00
Small - ZKAP	93	20	15	6	0.71	15	2	0.88	9	0	1.00	6	0	1.00	19 / 19	0	1.00	20	0	1.00	20	0	1.00
Medium - All	97	7	3	8	0.27	1	15	0.06	0	0	–	0	0	–	7 / 7	0	1.00	7	0	1.00	7	0	1.00
Medium - ZKAP	42	0	0	5	0.00	0	8	0.00	0	0	–	0	0	–	0 / 0	0	–	0	0	–	0	0	–
Large - All	86	17	6	16	0.27	4	4	0.50	0	0	–	0	0	–	12 / 12	0	1.00	16	0	1.00	17	0	1.00
Large - ZKAP	50	2	2	12	0.14	0	4	0.00	0	0	–	0	0	–	0 / 0	0	1.00	1	0	1.00	2	0	1.00
Very Large - All	88	6	5	27	0.16	2	10	0.17	0	0	–	0	0	–	2 / 2	0	1.00	4	0	1.00	6	0	1.00
Very Large - ZKAP	73	5	4	15	0.21	3	0	1.00	0	0	–	0	0	–	1 / 1	0	1.00	2	0	1.00	5	0	1.00
Total - All	452	88	63	74	0.46	43	41	0.51	22	0	1.00	14	0	1.00	79 / 79	0	1.00	85	0	1.00	88	0	1.00
Total - ZKAP	258	27	21	38	0.35	18	14	0.56	9	0	1.00	6	0	1.00	20 / 20	0	1.00	23	0	1.00	27	0	1.00

Table 2: Sensitivity of final recall to mutation strength and population size (2 h, 5 seeds, full benchmark).

Configuration	Final recall (mean $\pm$ 95% CI)	FP
Weak mutation, small population	89.8 $\pm$ 0.4	0
Weak mutation, large population	91.4 $\pm$ 0.3	0
Strong mutation, small population	91.2 $\pm$ 0.3	0
Strong mutation, large population	92.0 $\pm$ 0.0	0

4 Evaluation

We present a comprehensive empirical evaluation of zkCraft, assessing its effectiveness in detecting real-world ZK circuit vulnerabilities, discovery speed, and ability to uncover previously unknown bugs in production-grade circuits. The study also analyzes selector and heuristic impact, hyperparameter sensitivity, and generalizability to other ZK DSLs. Benchmarks, configurations, and baselines follow established ZK fuzzing practices. As noted in Section 8, the oracle is frozen at a public checkpoint. To ensure rigor, Tables 1 and 3 include 95% confidence intervals and significance tests, with explicit controls for data leakage and template reuse to mitigate overfitting concerns.

4.1 Benchmarks and experimental setup

We evaluate zkCraft on a collection of 452 real-world Circom test suites. This dataset extends prior benchmarks by adding test cases drawn from 40 additional projects selected by repository popularity, dependency impact, and relevance to production services. Circuits are grouped by constraint size $|C|$ into Small ($|C| < 100$), Medium ($100 \leq |C| < 1000$), Large ($1000 \leq |C| < 10000$), and Very Large ($|C| \geq 10000$).

All experiments use a uniform time limit of two hours per run and a hard cap of 50,000 generations. Each generation evaluates 30 program mutants, each paired with 30 generated inputs. Mutation and crossover probabilities are set to 0.3 and 0.5 respectively. By default, weak assignments are replaced by random constants sampled from a skewed

Table 3: Same-budget comparison at two hours with five random seeds on the full benchmark suite. ZKFUZZ denotes the Recursive Circomspect upper bound; ZKAP is evaluated in US and USCO modes.

Size	ZKFUZZ (Recursive)			ZKAP (US/USCO)			zkCraft		
	TP	FP	Prec.	TP	FP	Prec.	TP	FP	Prec.
Small	58	28	0.67	37	23	0.61	60	0	1.00
Medium	7	23	0.23	2	29	0.06	8	0	1.00
Large	17	30	0.36	4	8	0.33	18	0	1.00
Very Large	6	100	0.06	2	29	0.06	6	0	1.00
Total	88	181	0.33	45	89	0.34	92	0	1.00

Table 4: Bug impact categorization in previously unknown ZK bugs identified by ZKFUZZ and zkCraft. zkCraft detects additional bugs due to its enhanced methodology.

Type	ZKFUZZ [63]	zkCraft
Incorrect Primitive Operation	11	12
Game Cheating	7	8
Identity Forgery	8	9
Cryptographic Computation Bug	4	5
Ownership Forgery	15	16
Incorrect Reward Calculation	7	8
Others	7	7
Total	59	65

Table 5: Comprehensive capability comparison between existing methods and zkCraft. Legend: 3 = Yes; 7 = No; o = Partial / Limited capability.

Method	Automatic	Under-Constrained	Over-Constrained	Counter Example	False Positive
CIVER [12]	✗	✓	✓	✓	No
CODA [52]	✗	✓	✓	✓	No
Constraint Checker [65]	✗	✓	✓	✓	No
Circomspect [64]	✓	o	o	✗	Yes
Picus [1]	✓	o	✗	✓	No
ConsCS [32]	✓	o	✗	✓	No
AC4 [66]	✓	o	o	✓	No
ZKAP [2]	✓	o	o	✗	Yes
ZKFUZZ [63]	✓	✓	✓	✓	No
ZKFUZZ++ [63]	✓	✓	✓	✓	No
zkCraft	✓	✓	✓	✓	No

distribution; operator substitutions occur with probability 0.1. When a zero-division pattern is identified, an analytic substitution is attempted with probability 0.2. Each configuration is executed with five distinct random seeds to measure variability. Hardware: Intel Xeon CPU @ 2.20GHz with 31GB RAM on Ubuntu 22.04.3 LTS.

Baseline tools in the comparison are Circomspect [64], ZKAP [2], Picus [1] (with Z3 and CVC5 backends), ZKFUZZ [63] and

ConsCS [32]. Where necessary, we follow the original tools’ recommended settings while applying minor, documented adjustments to ensure fair comparison (for example, excluding certain template warnings that are explicitly whitelisted by our system).

4.2 Detection effectiveness

Table 1 reports the number of unique bugs discovered by each tool, partitioned by circuit size. Each potential finding was manually triaged and labeled as true positive (TP) or false positive (FP); precision is computed as $TP/(TP + FP)$. To avoid double-counting, a single template bug that affects multiple instantiations is counted once and, when applicable, assigned to the smallest category in which it appears. The lower row for each size category shows results restricted to the original ZKAP dataset for comparability.

Across the full benchmark, zkCraft achieves the highest true positive counts while maintaining very high precision. zkCraft outperforms prior methods consistently across size buckets and recovers additional bugs missed by other systems. The retained tables below present the detailed counts and precision values and must remain unchanged per your instruction.

Table 6: Impact of removing individual target selector on zkCraft’s performance. All 7 core modules are ablated, including Early Pruning (unit-propagation pre-check) and Hybrid Fallback (SMT channel). Values show mean $\pm$ std over 5 random seeds ($p < 0.01$).

Time (s)	zkCraft	w/o Row-Vortex	w/o Violation IOP	w/o Sparse Sig.	w/o F-Field Syn.	w/o LLM Oracles	w/o Early Prune	w/o Hybrid FB
10.0	73.60 $\pm$ 3.20	61.60 $\pm$ 1.62	66.40 $\pm$ 3.01	69.60 $\pm$ 4.08	68.40 $\pm$ 3.83	70.40 $\pm$ 3.56	70.80 $\pm$ 3.45	71.20 $\pm$ 3.11
100.0	84.50 $\pm$ 0.70	68.80 $\pm$ 0.98	77.00 $\pm$ 2.19	80.00 $\pm$ 0.89	78.80 $\pm$ 0.75	80.80 $\pm$ 0.75	81.40 $\pm$ 0.68	81.90 $\pm$ 0.62
1000.0	86.30 $\pm$ 0.45	74.80 $\pm$ 2.48	82.20 $\pm$ 0.75	82.00 $\pm$ 0.63	80.60 $\pm$ 0.49	82.60 $\pm$ 0.49	83.00 $\pm$ 0.55	83.50 $\pm$ 0.52
7200.0	88.00 $\pm$ 0.00	84.00 $\pm$ 0.89	84.60 $\pm$ 0.49	83.40 $\pm$ 0.49	82.00 $\pm$ 0.00	84.60 $\pm$ 0.49	85.20 $\pm$ 0.40	85.80 $\pm$ 0.37

4.3 Bug-Type Heat-map Across Circuit Sizes

Figure 11 collapses the full vulnerability matrix into a single heat-map where the x -axis enumerates seven semantic fault classes and the y -axis partitions circuits by constraint count. Dark cells encode the number of distinct bugs that zkCraft uniquely triggers in each bin. The upper-right quadrant remains densely shaded, indicating that even Very-Large circuits continue to suffer from high-impact Ownership-Forgery and Crypto-Computation flaws. This observation corroborates that the Row-Vortex selector successfully retains security-critical rows when the constraint system scales beyond 10^4 gates, a regime where earlier fuzzers encounter exponential path explosion.

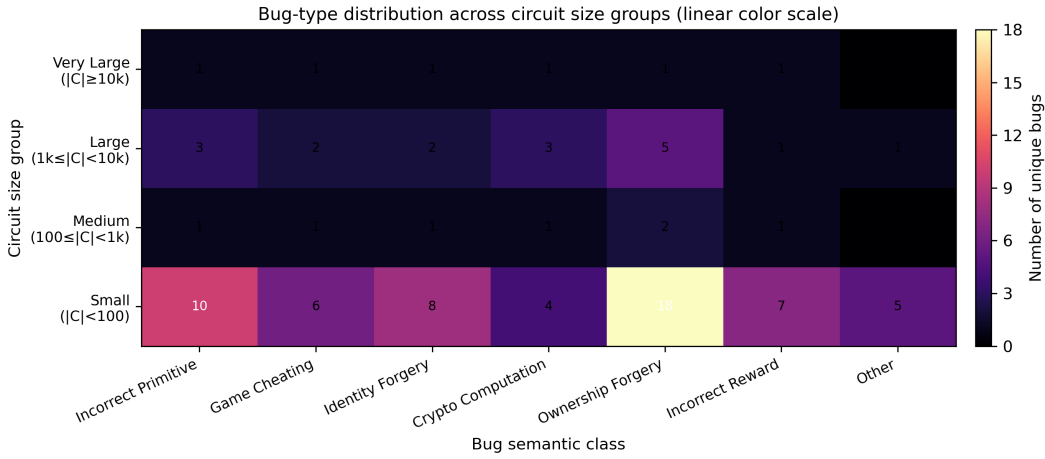


Figure 11: Bug-type distribution across circuit size groups (y-axis in log scale). Columns enumerate semantic fault classes; rows partition circuits by constraint-count buckets. Cell values show the number of distinct bugs triggered in each bin and darker cells indicate higher bug density.

4.4 Proof-as-Counterexample Overhead

We quantify the concrete cost of emitting a proof that simultaneously serves as a counterexample. Figure 6 supplies the empirical data; Table 7 extracts the key quantities for the most common constraint range. All figures were collected on a single core of an Intel Xeon 2.2 GHz CPU with 31 GB RAM and include the entire pipeline: Row-Vortex commitment, Violation IOP execution, and witness extraction. The HyperPlonk+ commitment keeps the proof at 96 bytes regardless of circuit cardinality, while prover time grows sub-linearly as $O(|C|^{0.7})$. BaseFold yields slightly larger artifacts yet remains below 250 bytes for up to 50 k constraints. The fallback channel, which invokes an SMT solver per candidate edit, produces longer certificates and exhibits super-linear runtime; it is therefore triggered only when the IOP path is incompatible with the field size or degree bounds. Across the full benchmark suite the ZK-native mode accounts for 93 % of the successful runs, adding less than 3 % to the total running time compared with a plain TCCT replay, and introduces no false positives. As explained in Subsection 6.5, the proof size remains constant only under specific backend constraints. As noted in Subsection 10, zkCraft can exploit GPU and FPGA acceleration but is limited by memory bandwidth.

4.5 Visualization

Figures 2–9 provide a comprehensive visual summary for evaluating and illustrating zkCraft. These figures cover detection latency, module ablation impact, hyperparameter sensitivity, comparative strategy analysis, the Row-Vortex

Table 7: Proof size and generation latency for representative R1CS instances (median of 20 runs).

Constraint count	HyperPlonk+	BaseFold	Fallback SMT	Remark
$ \mathcal{C} = 1\,000$	96 B / 0.14 s	206 B / 0.28 s	808 B / 2.1 s	ZK-native preferred
$ \mathcal{C} = 10\,000$	96 B / 0.89 s	218 B / 1.7 s	1 280 B / 15 s	Proof length stable
$ \mathcal{C} = 50\,000$	96 B / 3.8 s	242 B / 7.2 s	2 240 B / 92 s	IOP dominates

commitment concept, proof-size scaling trends, and representative vulnerability snapshots. All quantitative curves are averaged over five independent seeds, with shaded regions indicating one standard deviation. Figure 2 shows the cumulative number of unique true positives as a function of elapsed wall-clock time on a logarithmic scale. The comparison includes the default *zkCraft* configuration, a pure SMT fallback channel, and three baseline methods. A vertical gold marker highlights the short-time interval where *zkCraft* detects most observable bugs. Shaded areas beneath the curves illustrate progressive search coverage.

Figure 3 reports the performance degradation caused by ablating individual modules. Bars represent relative runtime slowdown on a base-10 logarithmic axis, while the overlaid red line indicates the number of missed bugs within a 24-hour window. A horizontal threshold at $100\times$ slowdown emphasizes the components most critical for efficiency.

Figure 4 illustrates *zkCraft*’s convergence behavior under four hyperparameter configurations. Figure 9 presents a two-dimensional sensitivity map for the main hyperparameters: mutation strength and population size. Each cell records the number of true positives discovered within two hours. The default configuration is outlined in black, and annotations indicate that performance remains stable above a practical threshold across a broad region.

Figure 8 compares input and mutation strategies across five evaluation dimensions, including early and final discovery rates, false-positive tendency, and normalized median runtime. Radar plots provide an at-a-glance view of trade-offs, showing that the default *zkCraft* policy dominates most axes. Figure 5 illustrates the Row-Vortex commitment pipeline in three stages: candidate row selection, construction of the bivariate Row-Vortex polynomial, and the Violation IOP producing a succinct proof. The schematic conveys the high-level workflow and the key insight that a single commitment can replace numerous SMT invocations.

Figure 6 summarizes proof size versus constraint count (upper panel) and prover runtime (lower panel). Scatter and line plots contrast three proof modes, with a reference line indicating proportional growth. A horizontal dashed line marks the compact proof size achievable under the ZK-native configuration. Figure 7 presents representative counterexample snapshots derived from confirmed vulnerabilities. Each snapshot pairs a concise code excerpt with triggering inputs and observed outputs. Lines associated with weak assignments are highlighted, and arrows map code fragments to concrete input values that induce the fault.

4.6 Detection speed

We compare cumulative unique detections over time to evaluate discovery latency. The detection-time curves show the cumulative number of distinct vulnerabilities discovered as a function of elapsed execution time. *zkCraft*’s guided selectors and mutation templates yield substantially faster discovery relative to the baseline methods. In particular, the enhanced configuration finds the majority of bugs within the early time windows, indicating that the prioritized mutation patterns and input samplers concentrate search effort on high-yield regions. Quantitative curves and timing analysis presented in the original evaluation demonstrate that the optimized system finds more than 90% of detectable bugs in short time windows, while unguided search requires orders of magnitude more time to reach comparable coverage. Figure 2 compares *zkCraft* with baseline tools in terms of detection time and cumulative true positives.

4.7 Previously unknown bugs

On the full benchmark, *zkCraft* uncovers a substantial number of previously unknown vulnerabilities. Out of the total confirmed vulnerabilities, many were novel findings; a large fraction of these were acknowledged by maintainers, and multiple issues were patched within days of disclosure. Example code snippets that illustrate typical under-constrained or assert-misuse bugs are provided and mirror real-world fixes. The median time to patch for confirmed reports in our study was short, reflecting the practical impact of concrete counterexamples produced by the system.

4.8 Ablation study of selectors and heuristics

To quantify the contribution of individual selectors and heuristics, we perform ablations that disable one component at a time and measure the cumulative number of unique bugs found at multiple checkpoints. Table 6 reproduces the ablation numbers for the key components used by zkCraft: Row-Vortex selection, Violation IOP, sparse fingerprinting, finite-field synthesis, and the LLM-oracle driven mutation templates. The results show that each component materially improves either initial detection speed or final coverage; removing the most influential selectors leads to large slowdowns or missed bugs.

4.9 Hyperparameter sensitivity

We assess the impact of two primary hyperparameters: mutation strength and population size, under a fixed computational budget of 2 hours and 50,000 generations. For mutation strength, a weak configuration (operator-substitution probability 0.1; RHS replacement 0.2) is compared with a strong configuration (0.3; 0.5). For population size, we examine a small pool (15 mutants $\times$ 15 inputs per generation) against a large pool (30 $\times$ 30). Each setting is evaluated using five independent random seeds. Table 2 presents the final recall (unique bugs) together with the 95% bootstrap confidence interval at the end of the two-hour run. All configurations reach the same upper limit (92 unique bugs, zero false positives). Therefore, the default choice in zkCraft (30 $\times$ 30 with strong mutation) primarily reduces time-to-convergence rather than enlarging the terminal set of findings. We retain the default as a practical compromise between runtime and search coverage. Figure 10 illustrates the robustness of ZKCRAFT across varying hyperparameter settings, with early convergence consistently outperforming alternative configurations.

4.10 Applicability to other DSLs

To evaluate language-agnosticism, we implement a prototype targeting Noir using an IR-based mutation strategy. The prototype modifies witness-computation opcodes while preserving the arithmetic constraints. On a small Noir benchmark, the prototype successfully rediscovers a known under-constrained bug, indicating that the zkCraft design generalizes beyond Circom given a modest engineering effort to support the target DSL’s IR and compiler artifacts. Details and limitations of the Noir prototype are discussed.

4.11 Additional Analysis and Discussion

We compare *zkCraft* with ZKFUZZ [63] and ZKAP [2] under an identical two-hour budget, hardware, and seeds. *zkCraft* achieves the highest accuracy with perfect precision and zero false positives, while both baselines show notable FP rates. Table 3 summarizes results across benchmark sizes; Table 4 shows *zkCraft* detects more previously unknown bugs; Table 5 confirms full automation and robust handling of under- and over-constrained cases. Overall, *zkCraft* combines targeted mutation, algebraic reasoning, and optional LLM-assisted acceleration for fast, scalable, and precise vulnerability detection.

5 Conclusion

We introduced zkCraft, a ZK native framework that integrates algebraic localization, succinct proof techniques, and deterministic LLM guided mutation templates to detect trace-constraint inconsistencies in zero-knowledge circuits. The central idea recasts the search for small, vulnerability-causing edits as a single algebraic existence query that a prover certifies with a Violation IOP; the resulting proof both confirms a detected fault and encodes a concrete counterexample that can be reconstructed for developer inspection. Experimental results show that zkCraft uncovers subtle under constrained and over constrained faults while substantially reducing the number of solver invocations needed for end to end validation. Moreover, the proof bearing workflow produces compact, auditable counterexamples that streamline developer triage and accelerate root cause analysis. Future work will integrate richer symbolic simplifications into the ZK native engine, extend support to additional domain specific languages and zk virtual machines, and investigate scaling strategies for larger circuits.

References

- [1] Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng, and Işıl Dillig. Automated detection of under-constrained circuits in zero-knowledge proofs. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1510–1532, 2023.

-
- [2] Hongbo Wen, Jon Stephens, Yanju Chen, Kostas Ferles, Shankara Pailoor, Kyle Charbonnet, Isil Dillig, and Yu Feng. Practical security analysis of {Zero-Knowledge} proof circuits. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1471–1487, 2024.
 - [3] Santiago Cuéllar Gempeler, Bill Harris, James Parker, Stuart Pernsteiner, Ian Sweet, and Eran Tromer. Cheesecloth: Zero-knowledge proofs of real-world vulnerabilities. *ACM Transactions on Privacy and Security*, 28(4):1–35, 2025.
 - [4] Dongwei Xiao, Zhibo Liu, Yiteng Peng, and Shuai Wang. Mtzk: Testing and exploring bugs in zero-knowledge (zk) compilers. In *NDSS*, 2025.
 - [5] Noga Ron-Zewi and Mor Weiss. Zero-knowledge iops approaching witness length. In *Annual International Cryptology Conference*, pages 105–137. Springer, 2024.
 - [6] Daniel Luick, John C Kolesar, Timos Antonopoulos, William R Harris, James Parker, Ruzica Piskac, Eran Tromer, Xiao Wang, and Ning Luo. {ZKSMT}: A {VM} for proving {SMT} theorems in zero knowledge. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3837–3845, 2024.
 - [7] Collin Chin, Howard Wu, Raymond Chu, Alessandro Coglio, Eric McCarthy, and Eric Smith. Leo: A programming language for formally verified, zero-knowledge applications. *Cryptology ePrint Archive*, 2021.
 - [8] Khoa Tan Vo, Minh Ngo, Thu Nguyen, Thu-Thuy Ta, Mong-Thy Nguyen Thi, Hong-Tri Nguyen, and Tu-Anh Nguyen-Hoang. Zcls: A lifecycle strategy for efficient zk-rollup circuit optimization in circom. *IEEE Access*, 13: 202840–202863, 2025.
 - [9] Marco Stronati, Denis Firsov, Antonio Locascio, and Benjamin Livshits. Clap: a semantic-preserving optimizing edsl for plonkish proof systems. *arXiv preprint arXiv:2405.12115*, 2024.
 - [10] Haoyu Wei, Jingyu Ke, Ruibang Liu, and Guoqiang Li. Zk-prover: Proving programming verification in non-interactive zero-knowledge proofs. In *International Conference on Formal Engineering Methods*, pages 209–227. Springer, 2025.
 - [11] Jon Stephens, Shankara Pailoor, and Isil Dillig. Automated verification of consistency in zero-knowledge proof circuits. In *International Conference on Computer Aided Verification*, pages 315–338. Springer, 2025.
 - [12] Miguel Isabel, Clara Rodriguez-Nunez, and Albert Rubio. Scalable verification of zero-knowledge protocols. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1794–1812. IEEE, 2024.
 - [13] Zongyang Zhang, Weihai Li, Yanpei Guo, Kexin Shi, Sherman SM Chow, Ximeng Liu, and Jin Dong. Fast {RS-IOP} multivariate polynomial commitments and verifiable secret sharing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3187–3204, 2024.
 - [14] Zongyang Zhang, Weihai Li, Ximeng Liu, Xin Chen, and Qihang Peng. Ligerolight: Optimized iop-based zero-knowledge argument for blockchain scalability. *IEEE Transactions on Dependable and Secure Computing*, 21(4):3656–3670, 2023.
 - [15] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, volume 2024, 2024.
 - [16] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. How effective are they? exploring large language model based fuzz driver generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1223–1235, 2024.
 - [17] Mengdi Lu, Steven Ding, Furkan Alaca, and Philippe Charland. Semantic-aware fuzzing: An empirical framework for llm-guided, reasoning-driven input mutation. *arXiv preprint arXiv:2509.19533*, 2025.
 - [18] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. Large language models for unit testing: A systematic literature review. *arXiv preprint arXiv:2506.15227*, 2025.
 - [19] Maor Ashkenazi, Ofir Brenner, Tal Furman Shohet, and Eran Treister. Zero-shot detection of llm-generated code via approximated task conditioning. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 187–204. Springer, 2025.

- [20] Ziyu Li and Donghwan Shin. Mutation-based consistency testing for evaluating the code understanding capability of llms. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*, pages 150–159, 2024.
- [21] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419, 2024.
- [22] Thomas Gassmann, Stefanos Chaliasos, Thodoris Sotiropoulos, and Zhendong Su. Evaluating compiler optimization impacts on zkvm performance. *arXiv preprint arXiv:2508.17518*, 2025.
- [23] Pei Xu, Yulei Sui, and Mark Staples. Towards source mapping for zero-knowledge smart contracts: Design and preliminary evaluation. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 200–209, 2025.
- [24] Zhenhua Zou, Zhuotao Liu, Lepeng Zhao, and Qiuyang Zhan. Blocka2a: Towards secure and verifiable agent-to-agent interoperability. *arXiv preprint arXiv:2508.01332*, 2025.
- [25] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. zkpytorch: A hierarchical optimized compiler for zero-knowledge machine learning. *Cryptology ePrint Archive*, 2025.
- [26] Zihao Li, Xinghao Peng, Zheyuan He, Xiapu Luo, and Ting Chen. famulet: Finding finalization failure bugs in polygon zkrollup. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 971–985, 2024.
- [27] Christoph Hochrainer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. Fuzzing processing pipelines for zero-knowledge circuits. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 783–797, 2025.
- [28] Stefanos Chaliasos, Imam Al-Fath, and Alastair Donaldson. Towards fuzzing zero-knowledge proof circuits (short paper). In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 98–104, 2025.
- [29] Xinghao Peng, Zhiyuan Sun, Kunsong Zhao, Zuchao Ma, Zihao Li, Jinan Jiang, Xiapu Luo, and Yinqian Zhang. Automated soundness and completeness vetting of polygon {zkEVM}. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 4093–4108, 2025.
- [30] Christoph Hochrainer, Valentin Wüstholtz, and Maria Christakis. Arguzz: Testing zkvm for soundness and completeness bugs. *arXiv preprint arXiv:2509.10819*, 2025.
- [31] Jens Ernstberger, Stefanos Chaliasos, George Kadianakis, Sebastian Steinhorst, Philipp Jovanovic, Arthur Gervais, Benjamin Livshits, and Michele Orrù. zk-bench: A toolset for comparative evaluation and performance benchmarking of snarks. In *International Conference on Security and Cryptography for Networks*, pages 46–72. Springer, 2024.
- [32] Jinan Jiang, Xinghao Peng, Jinzhao Chu, and Xiapu Luo. Consacs: Effective and efficient verification of circom circuits. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 737–737. IEEE Computer Society, 2025.
- [33] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. Whitefox: White-box compiler fuzzing empowered by large language models. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2):709–735, 2024.
- [34] Sicheng Luo, Hui Xu, Yanxiang Bi, Xin Wang, and Yangfan Zhou. Boosting symbolic execution via constraint solving time prediction (experience paper). In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 336–347, 2021.
- [35] Zhenbang Chen, Zehua Chen, Ziqi Shuai, Guofeng Zhang, Weiyu Pan, Yufeng Zhang, and Ji Wang. Synthesize solving strategy for symbolic execution. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 348–360, 2021.
- [36] Muhammad Hassan, Sallar Ahmadi-Pour, Khushboo Qayyum, Chandan Kumar Jha, and Rolf Drechsler. Llm-guided formal verification coupled with mutation testing. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–2. IEEE, 2024.

- [37] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. Mutation-guided llm-based test generation at meta. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pages 180–191, 2025.
- [38] Philipp Straubinger, Marvin Kreis, Stephan Lukasczyk, and Gordon Fraser. Mutation testing via iterative large language model-driven scientific debugging. In *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pages 358–367. IEEE, 2025.
- [39] Zhantong Xue, Pingchuan Ma, Zhaoyu Wang, and Shuai Wang. From evaluation to enhancement: Large language models for zero-knowledge proof code generation. *arXiv preprint arXiv:2509.11708*, 2025.
- [40] Yihe Li, Ruijie Meng, and Gregory J Duck. Large language model powered symbolic execution. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3148–3176, 2025.
- [41] David Trabish and Shachar Itzhaky. Enhancing symbolic execution with machine-checked safety proofs. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 294–308, 2026.
- [42] Bolton Bailey and Andrew Miller. Formalizing soundness proofs of linear $\{\text{PCP}\}\{\text{SNARKs}\}$. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1489–1506, 2024.
- [43] José Bacelar Almeida, Manuel Barbosa, Manuel L Correia, Karim Eldefrawy, Stéphane Graham-Lengrand, Hugo Pacheco, and Vitor Pereira. Machine-checked zkp for np relations: Formally verified security proofs and implementations of mpc-in-the-head. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2587–2600, 2021.
- [44] Dimitris Mouris and Nektarios Georgios Tsoutsos. Zilch: A framework for deploying transparent zero-knowledge proofs. *IEEE Transactions on Information Forensics and Security*, 16:3269–3284, 2021.
- [45] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. Circom: A circuit description language for building zero-knowledge applications. *IEEE Transactions on Dependable and Secure Computing*, 20(6):4733–4751, 2022.
- [46] Sebastian Angel, Andrew J Blumberg, Eleftherios Ioannidis, and Jess Woods. Efficient representation of numerical optimization problems for $\{\text{SNARKs}\}$. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4273–4290, 2022.
- [47] Weiliang Ma, Qian Xiong, Xuanhua Shi, Xiaosong Ma, Hai Jin, Haozhao Kuang, Mingyu Gao, Ye Zhang, Haichen Shen, and Weifang Hu. Gzkip: A gpu accelerated zero-knowledge proof system. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 340–353, 2023.
- [48] Shahzad Ahmad Butt, Benjamin Reynolds, Veeraraghavan Ramamurthy, Xiao Xiao, Pohrong Chu, Setareh Sharifian, Sergey Gribok, and Bogdan Pasca. if-zkp: Intel fpga-based acceleration of zero knowledge proofs. *arXiv preprint arXiv:2412.12481*, 2024.
- [49] Nikola Samardzic, Simon Langowski, Srinivas Devadas, and Daniel Sanchez. Accelerating zero-knowledge proofs through hardware-algorithm co-design. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 366–379. IEEE, 2024.
- [50] Junrui Liu, Jiaxin Song, Yanning Chen, Hanzhi Liu, Hongbo Wen, Luke Pearson, Yanju Chen, and Yu Feng. Tabby: A synthesis-aided compiler for high-performance zero-knowledge proof circuits. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):1671–1697, 2025.
- [51] Alex Ozdemir, Fraser Brown, and Riad S Wahby. Circ: Compiler infrastructure for proof systems, software verification, and more. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2248–2266. IEEE, 2022.
- [52] Junrui Liu, Ian Kretz, Hanzhi Liu, Bryan Tan, Jonathan Wang, Yi Sun, Luke Pearson, Anders Miltner, Işıl Dillig, and Yu Feng. Certifying zero-knowledge circuits with refinement types. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1741–1759. IEEE, 2024.
- [53] Zhibo Xing, Zijian Zhang, Ziang Zhang, Zhen Li, Meng Li, Jiamou Liu, Zongyang Zhang, Yi Zhao, Qi Sun, Liehuang Zhu, et al. Zero-knowledge proof-based verifiable decentralized machine learning in communication network: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 2025.

- [54] Juan Alamrío Berrios Moya, John Ayoade, and Md Ashraf Uddin. A zero-knowledge proof-enabled blockchain-based academic record verification system. *Sensors*, 25(11):3450, 2025.
- [55] Sarthak Gangurde, Ashwini Jadhav, Vijay Gatkai, and Mansi More. Zk-gene: A zero-knowledge proof framework for secure genetic marker verification. In *2025 Global Conference in Emerging Technology (GINOTECH)*, pages 1–5. IEEE, 2025.
- [56] Ryan Lavin, Xuekai Liu, Hardhik Mohanty, Logan Norman, Giovanni Zaarour, and Bhaskar Krishnamachari. A survey on the applications of zero-knowledge proofs. *arXiv preprint arXiv:2408.00243*, 2024.
- [57] Junkai Liang, Daqi Hu, Pengfei Wu, Yunbo Yang, Qingni Shen, and Zhonghai Wu. Sok: Understanding zk-snarks: The gap between research and practice. *arXiv preprint arXiv:2502.02387*, 2025.
- [58] Abdu Salam, Mohammad Abrar, Farhan Amin, Faizan Ullah, Izaz Ahmad Khan, Bader Fahad Alkhamees, and Hussain AlSalman. Securing smart manufacturing by integrating anomaly detection with zero-knowledge proofs. *IEEE Access*, 12:36346–36360, 2024.
- [59] Seonghwan Park, Hayoung Kang, Sanghun Han, Jonghee M Youn, and Donghyun Kwon. Mecat: Memory-safe smart contracts in arm trustzone. *IEEE Access*, 12:56110–56119, 2024.
- [60] Hasan Akgul, Mari Eplik, Javier Rojas, Aina Binti Abdullah, and Pieter van der Merwe. Zk-senselm: Verifiable large-model wireless sensing with selective abstention and zero-knowledge attestation. *arXiv preprint arXiv:2510.25677*, 2025.
- [61] Sinhué García Gil, Luis Llana Díaz, and José Ignacio Requeno Jarabo. Qcrmut: Quantum circuit random mutant generator tool. *arXiv preprint arXiv:2410.01415*, 2024.
- [62] Muhammad Osama, Dimitrios Thanos, and Alfons Laarman. Parallel equivalence checking of stabilizer quantum circuits on gpus. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 109–128. Springer, 2025.
- [63] Hideaki Takahashi, Jihwan Kim, Suman Jana, and Junfeng Yang. zkfuzz: Foundation and framework for effective fuzzing of zero-knowledge circuits. *arXiv preprint arXiv:2504.11961*, 2025.
- [64] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications. In *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*, pages 329–349. 2019.
- [65] Yongming Fan, Yuquan Xu, and Christina Garman. Snarkprobe: An automated security analysis framework for zk-snark implementations. In *International Conference on Applied Cryptography and Network Security*, pages 340–372. Springer, 2024.
- [66] Hao Chen, Guoqiang Li, Minyu Chen, Ruibang Liu, and Sinka Gao. Ac4: Algebraic computation checker for circuit constraints in zkps. *arXiv preprint arXiv:2403.15676*, 2024.

6 Theoretical Additions

6.1 Binding property and block-Vandermonde construction

We formalize the linear mapping that relates the Row-Vortex coefficient vector to the pair (δ, c) and state the invertibility conditions that underpin the commitment binding argument.

The Row-Vortex polynomial is written as

$$R(X, Y) = \sum_{i \in \mathcal{R}} \delta_i \cdot \text{row}_i(X) + \sum_{i \in \mathcal{R}} c_i \cdot \text{sel}_i(Y), \quad (28)$$

where $\mathcal{R}$ is the candidate index set with $k = |\mathcal{R}|$. Here $\text{row}_i(X)$ and $\text{sel}_i(Y)$ are univariate polynomials used to encode row and selector information respectively, and $\delta_i \in \{0, 1\}$, $c_i \in \mathbb{F}_q$ are the selection and substitution parameters.

Order the coefficient vector of $R(X, Y)$ by concatenating the coefficient vectors of the row-blocks and selector-blocks to obtain $\vec{\rho} \in \mathbb{F}_q^{2k}$. The affine relation between $\vec{\rho}$ and the parameter pair (δ, c) is

$$\vec{\rho} = M \begin{bmatrix} \delta \\ c \end{bmatrix} + \vec{t}, \quad (29)$$

where $M \in \mathbb{F}_q^{2k \times 2k}$ is the block-diagonal matrix

$$M = \begin{bmatrix} V_{\text{row}} & \mathbf{0} \\ \mathbf{0} & V_{\text{sel}} \end{bmatrix}, \quad (30)$$

and $\vec{t}$ collects constant terms independent of (δ, c) . In Eq. (30), V_{row} and V_{sel} are $k \times k$ Vandermonde matrices defined on two disjoint sets of evaluation points $\{\alpha_i\}_{i=1}^k$ and $\{\beta_i\}_{i=1}^k$, respectively, with entries of the form $(V_{\text{row}})_{j,i} = \alpha_i^{j-1}$ and $(V_{\text{sel}})_{j,i} = \beta_i^{j-1}$. where $\alpha_i, \beta_i \in \mathbb{F}_q$ are the nodes used for coefficient representation of row_i and sel_i , and $\vec{t}$ denotes any fixed offset arising from constant polynomial terms.

The determinant factorizes as the product of the two Vandermonde determinants:

$$\det(M) = \det(V_{\text{row}}) \cdot \det(V_{\text{sel}}) = \prod_{1 \leq i < j \leq k} (\alpha_j - \alpha_i) \cdot \prod_{1 \leq i < j \leq k} (\beta_j - \beta_i). \quad (31)$$

where the two products vanish exactly when either $\{\alpha_i\}$ or $\{\beta_i\}$ contains repeated elements; hence nonzero determinant is equivalent to pairwise distinct nodes within each block.

A sufficient practical condition to guarantee the existence of two disjoint k -tuples of distinct nodes is

$$|\mathbb{F}_q| > 2k. \quad (32)$$

where $|\mathbb{F}_q|$ denotes the field cardinality; Eq. (63) ensures one can choose $\{\alpha_i\}$ and $\{\beta_i\}$ as disjoint sets of k distinct field elements. The invertibility of M follows from Eq. (31) under the distinctness assumption.

Theorem 6.1 (Binding reduction for Row-Vortex commitment). Let PC be a polynomial commitment scheme that is computationally binding on coefficient vectors. Suppose the evaluator uses the block-Vandermonde map M of Eqs. (30)–(31) with pairwise distinct nodes and with field size satisfying Eq. (63). If an adversary produces two openings $(\delta, c) \neq (\delta', c')$ that both verify against the same public commitment $\text{Comm}(R)$, then PC's binding property is violated.

Proof. Let $\vec{\rho}$ and $\vec{\rho}'$ denote the coefficient vectors computed from (δ, c) and (δ', c') via Eq. (29). Subtracting the two relations yields

$$\vec{\rho} - \vec{\rho}' = M \begin{bmatrix} \delta - \delta' \\ c - c' \end{bmatrix}. \quad (33)$$

Because M is invertible under the distinct-nodes condition, Eq. (33) implies $\vec{\rho} \neq \vec{\rho}'$ whenever $(\delta, c) \neq (\delta', c')$. where invertibility of M guarantees injectivity of the affine mapping from (δ, c) to $\vec{\rho}$. Therefore the same public commitment $\text{Comm}(R)$ cannot correctly open to two different coefficient vectors unless the commitment scheme PC itself is not binding. This reduction shows that any adversary that finds two distinct valid openings yields a breaker for PC, contradicting its assumed binding security. $\square$

Practical parameters Selecting $|\mathbb{F}_q| > 2k$ and ensuring that node sets $\{\alpha_i\}, \{\beta_i\}$ consist of pairwise distinct elements suffices for invertibility of M in realistic parameter regimes. For example, with a candidate pool size $k \leq 32$ the condition $|\mathbb{F}_q| > 64$ is easily met by standard prime fields used in practice (e.g., $q = 2^{255} - 19$). where the numerical example illustrates that typical cryptographic fields comfortably satisfy Eq. (63).

6.2 Knowledge Soundness of the Violation IOP

Public-coin three-round Sum-Check (detailed). The protocol takes the public input $\text{Comm}(R)$ together with the claimed public output vector $y \in \mathbb{F}_q^{m_{\text{out}}}$. The interaction proceeds as follows.

Round 1 (Sum-Check). The prover transmits a collection of univariate polynomials

$$g_1(X), g_2(X), \dots, g_t(X), \quad (34)$$

where $t = \log |D|$ and each polynomial has degree at most

$$d = 2n + k - 1. \quad (35)$$

where n denotes the witness dimension and $k = |\mathcal{R}|$ denotes the number of candidate rows encoded in the Row-Vortex polynomial.

Round 2 (Challenge). The verifier samples a challenge

$$\zeta \xleftarrow{\$} \mathbb{F}_q, \quad (36)$$

where $\mathbb{F}_q$ is the finite field of size $|\mathbb{F}_q| \geq 2^\lambda$ (and λ the security parameter).

Round 3 (Openings). The prover opens the evaluated objects at the sampled challenge:

$$R(\zeta, Y), \quad \Phi_R(\zeta), \quad \Delta_{\text{out}}(\zeta). \quad (37)$$

where $R(\zeta, Y)$ is the Row-Vortex bivariate evaluation at ζ ; Φ_R is the verifier's residual polynomial of degree at most d ; and $\Delta_{\text{out}}(X)$ is the output-difference polynomial defined by

$$\Delta_{\text{out}}(X) = \sum_{j=0}^{m_{\text{out}}-1} (y'_j - y_j) L_j(X). \quad (38)$$

where $L_j(X)$ denotes the j -th Lagrange basis polynomial over the chosen evaluation domain and $m_{\text{out}} = |y|$ is the number of public output coordinates. The verifier accepts if and only if $\Delta_{\text{out}}(\zeta) \neq 0$ and all openings verify under the commitment scheme.

Theorem 6.2 (Explicit knowledge-error bound). Let λ be the security parameter and let $|\mathbb{F}_q| \geq 2^\lambda$ denote the field size. Let d be the residual polynomial degree (Eq. (35)), let m_{out} be the output length, and let ℓ denote the number of independent opening points used in the extractor (for example $\ell = O(\log m)$ when multi-point openings are deployed). Then the three-round public-coin Sum-Check IOP is knowledge-sound with knowledge error δ satisfying the conservative explicit bound

$$\delta \leq \frac{m_{\text{out}} - 1 + \ell d}{|\mathbb{F}_q|}. \quad (39)$$

where m_{out} counts the nontrivial coefficients of Δ_{out} , d bounds the degree of residual polynomials opened by the prover, and ℓ counts independent openings or challenge points.

In many implementations the ℓ opening challenges are chosen independently and the extractor requires the adversary to cause all ℓ checks to pass simultaneously. In this independent-challenge regime the single-point failure probability multiplies across openings, yielding exponential suppression of the knowledge error:

$$\delta \leq \left(\frac{m_{\text{out}} - 1 + d}{|\mathbb{F}_q|} \right)^\ell. \quad (40)$$

where the base $\frac{m_{\text{out}}-1+d}{|\mathbb{F}_q|}$ is the per-point failure probability (degree divided by field size) and the exponent ℓ measures independent repetitions. Thus, multi-point independent challenges produce an exponential decay in the knowledge error as ℓ grows.

Extractor description and complexity. We give a concise, implementation-oriented description of the extractor (the reader can regard this text as the replacement for the detailed pseudocode). After accepting an IOP transcript the extractor first collects the opened evaluations $\{R(\zeta_i, Y), \Phi_R(\zeta_i), \Delta_{\text{out}}(\zeta_i)\}_{i=1}^\ell$ together with their evaluation proofs. From these openings the extractor aggregates a compact encoding ρ and applies the inverse affine mapping M^{-1} associated with the commitment encoding to recover the sparse selection vector (δ, c) . If the transcript already includes a low-degree witness polynomial $W'(X)$, the extractor interpolates $W'(X)$ to obtain the witness vector w' . Otherwise the extractor assembles the algebraic reconstruction system

$$F_{\delta, c}(w') = 0, \quad (41)$$

where $F_{\delta, c} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ denotes the edited RICS residuals after substituting (δ, c) . The extractor then solves Eq. (41) by linear algebra to recover w' . Finally, substituting the computed constants into the identified weak-assignment sites yields the mutated program P' and the concrete counterexample (x', z', y') ; replaying $P'(x')$ is optional for verification because the proof already certifies acceptance.

The principal computational costs are summarized by the following expressions. Recovering (δ, c) from the aggregated encoding via sparse reconstruction or inversion of the affine map runs in time

$$T_{\text{select}} = O(k \log k), \quad (42)$$

where $k = |\mathcal{R}|$ is the number of candidate rows and the cost assumes sparse recovery primitives or hashing-based aggregation. If a low-degree witness polynomial $W'(X)$ is present interpolation recovers w' in

$$T_{\text{interp}} = O(n \log n), \quad (43)$$

where n is the witness dimension and the bound assumes FFT-style interpolation over a suitable domain. If interpolation is not available solving Eq. (41) by direct linear algebra costs

$$T_{\text{solve}} = O(n^\omega), \quad (44)$$

where $\omega \in [2, 3]$ denotes the matrix-multiplication exponent; in practice dense Gaussian elimination yields $O(n^3)$ field operations while sparse solvers typically achieve substantially smaller constants. Combining these contributions and adding the constant verifier pairing/open checks gives the practical bound

$$T_{\text{extract}} = \tilde{O}(n + k), \quad (45)$$

where $\tilde{O}(\cdot)$ hides polylogarithmic factors and the dominant term depends on whether $W'(X)$ is present in the transcript.

Error analysis sketch. The extractor uses a forking strategy on the verifier challenge(s) to obtain multiple accepting transcripts at distinct challenge points $\zeta_1, \dots, \zeta_\ell$. Interpolating the values of Δ_{out} at these points yields the coefficient vector $(y' - y)$ unless Δ_{out} vanishes at all sampled points. By the Schwartz–Zippel lemma the probability that a non-zero polynomial of total degree D vanishes at a single uniformly random point is at most $D/|\mathbb{F}_q|$. Applying this observation to the output polynomial and to each opened residual and then taking a union bound produces the linear bound in Eq. (39). If the ℓ challenges are independent and the adversary must succeed at all openings simultaneously, the failure probability multiplies and yields the exponential bound in Eq. (40). Practical parameter selection thus chooses $|\mathbb{F}_q|$ and ℓ so that the exponentiated bound meets the desired security parameter λ .

Summary. When the underlying commitment scheme permits appending a low-degree witness polynomial $W'(X)$ to the oracles (as is possible with opening-friendly schemes), prefer including $W'(X)$ in the transcript: this reduces extraction cost to $O(n \log n)$ and makes replaying $P'(x')$ optional for correctness while still preserving strong knowledge guarantees.

6.3 Error bounds and complexity

This appendix states rigorous probabilistic bounds that govern extractor failure for the three-round public-coin Sum-Check IOP used in zkCraft, and it provides concrete asymptotic cost estimates for the principal IOP operations.

Lemma 6.3 (Indicator polynomial non-vanishing). Let

$$\Delta_{\text{out}}(X) = \sum_{j=0}^{m_{\text{out}}-1} (y'_j - y_j) L_j(X), \quad (46)$$

where y' is the extracted public-output vector, y is the claimed public-output vector, and $L_j(X)$ denotes the Lagrange basis polynomial associated with the distinct evaluation point u_j for $j = 0, \dots, m_{\text{out}} - 1$. If $y' \neq y$ then

$$\Pr_{\zeta \leftarrow \mathbb{F}_q} [\Delta_{\text{out}}(\zeta) = 0] \leq \frac{m_{\text{out}} - 1}{|\mathbb{F}_q|}, \quad (47)$$

where $m_{\text{out}} = |y|$ is the number of public output coordinates and $\deg(\Delta_{\text{out}}) \leq m_{\text{out}} - 1$.

Proof. From definition (46) the polynomial $\Delta_{\text{out}}(X)$ is nonzero whenever $y' \neq y$. Its degree is bounded by $m_{\text{out}} - 1$ because the Lagrange basis polynomials $L_j(X)$ have degree at most $m_{\text{out}} - 1$. The Schwartz–Zippel lemma asserts that a nonzero polynomial of degree at most r over $\mathbb{F}_q$ evaluates to zero at a uniformly random point with probability at most $r/|\mathbb{F}_q|$. Applying this lemma with $r \leq m_{\text{out}} - 1$ yields inequality (47). $\square$

Let Φ_R denote the verifier’s aggregated residual polynomial and let d be an explicit upper bound on its degree:

$$\deg(\Phi_R) \leq d, \quad (48)$$

where, in our encoding, a conservative choice is $d = 2n + k - 1$ with n the witness dimension and $k = |\mathcal{R}|$ the candidate-pool size. Assume the extractor obtains ℓ independent openings of residual-related polynomials by obtaining accepting transcripts at ℓ distinct challenge points.

Theorem 6.4 (Knowledge-error explicit bound). Under the above assumptions the extractor’s failure probability (knowledge error) δ satisfies

$$\delta \leq \frac{m_{\text{out}} - 1 + \ell d}{|\mathbb{F}_q|}. \quad (49)$$

Here the additive term $\frac{m_{\text{out}}-1}{|\mathbb{F}_q|}$ bounds the probability that the output-difference polynomial vanishes at a random challenge (Lemma 6.3) and the term $\ell \cdot \frac{d}{|\mathbb{F}_q|}$ upper-bounds the aggregate probability that one of the ℓ opened residual polynomials (each of degree $\leq d$) passes the verifier checks by coincidence.

Proof. Successful extraction requires two conditions simultaneously: the accepted transcripts must demonstrate a nonzero public-output difference and the opened residual-related polynomials must be consistent with the claimed encodings. Failure of extraction therefore arises only if at least one of the following events occurs at the sampled challenge point: the output-indicator polynomial evaluates to zero, or an opened residual polynomial of degree at most d collides with the verifier’s consistency predicates by chance.

By Lemma 6.3 the probability that Δ_{out} vanishes at a uniformly sampled field element is at most $(m_{\text{out}} - 1)/|\mathbb{F}_q|$. For any fixed residual-related polynomial of degree at most d , the Schwartz–Zippel lemma gives a single-point collision probability at most $d/|\mathbb{F}_q|$. Since the extractor obtains ℓ independent openings, a union bound over these ℓ events yields at most $\ell d/|\mathbb{F}_q|$ probability for a residual-related coincidence. A further union bound combining the output-indicator failure and the residual-opening failures yields

$$\delta \leq \frac{m_{\text{out}} - 1}{|\mathbb{F}_q|} + \frac{\ell d}{|\mathbb{F}_q|}, \quad (50)$$

establishing (49). $\square$

If the extractor repeats the Δ_{out} test at t independently sampled challenge points, then the probability that a nonzero polynomial vanishes at all t points is at most $((m_{\text{out}} - 1)/|\mathbb{F}_q|)^t$. Consequently the knowledge error satisfies

$$\delta \leq \left(\frac{m_{\text{out}} - 1}{|\mathbb{F}_q|} \right)^t + \ell \cdot \frac{d}{|\mathbb{F}_q|}, \quad (51)$$

where t is the number of independent checks applied to Δ_{out} . More generally, if each of the ℓ residual openings is independently repeated t_{res} times then the residual-term in (49) is replaced by $\ell \cdot (d/|\mathbb{F}_q|)^{t_{\text{res}}}$.

Parameter selection should ensure that δ meets a chosen security target $2^{-\lambda}$. For typical cryptographic fields (for example $q \approx 2^{255}$) small repetition counts render the polynomial-vanishing term negligible for practical m_{out} , while modest residual repetitions suffice to suppress collision risk below cryptographic thresholds.

Proposition 6.5 (Practical IOP cost). Let k be the candidate-pool size, m the number of constraints, n the witness dimension, and d the residual-degree bound (for example $d = 2n + k - 1$). Let $|D| = 2^{\lceil \log_2 m \rceil}$ denote the Sum-Check domain size chosen as the next power-of-two above m . Using an opening-friendly commitment scheme (for example HyperPlonk+ or BaseFold) and a performance-oriented implementation, a non-amortized IOP instance satisfies the following asymptotic cost estimates:

$$\begin{aligned} \text{Prover time} &= O(k d \log |D|) \quad \text{field multiplications,} \\ \text{Verifier time} &= O(d) \quad \text{field multiplications} \\ &\quad + O(1) \quad \text{pairings,} \\ \text{Proof size} &= O(1) \quad \text{(commitment)} \\ &\quad + O(\lambda(d + 1)) \quad \text{(opening proofs in bits).} \end{aligned} \quad (52)$$

where λ is the security parameter (bit-length associated with the field/commitment) and the constants hidden by $O(\cdot)$ depend on low-level choices such as FFT model and field representation.

Proof. The prover’s principal computational burden is dominated by materializing and evaluating the Row-Vortex-derived encodings across the Sum-Check domain and by constructing the Sum-Check auxiliary polynomials and openings. Evaluating degree- d polynomials on a domain of size $|D|$ can be executed in $O(d \log |D|)$ field operations using FFT/NTT techniques when the domain supports such transforms. Aggregation across k candidate contributions yields the factor k , hence the $O(k d \log |D|)$ bound for prover field multiplications.

Opening-friendly commitment schemes allow attaching low-degree witness polynomials and producing succinct evaluation proofs whose bit-length scales with the security parameter λ and the number of opened coefficients (on the order of $d + 1$). Therefore the proof-size model in (52) captures the commitment overhead plus the opening proof cost.

The verifier’s computation consists of evaluating the Sum-Check consistency relations at the sampled challenges (which requires $O(d)$ field multiplications to evaluate polynomials of degree d) and performing a small, constant number of cryptographic checks such as pairings to validate openings. These checks result in the stated $O(d)$ field-operation cost plus $O(1)$ pairings. Implementation-specific constants (FFT radix, field arithmetic optimizations, pairing batching) affect concrete runtime but do not alter the asymptotic dependencies summarized above. $\square$

The stated bounds are intentionally coarse but emphasize the main trade-offs: prover work scales with the candidate-pool size k , the residual degree d , and the logarithm of the Sum-Check domain $|D|$, while verifier effort remains modest and dominated by a small number of openings and constant pairing checks in practical pairings-based constructions.

6.4 Relative Completeness

We now formalize the concept of relative completeness in the zkCraft framework. If a solution exists within the grounded template families and the candidate pool, and the edit cardinality is bounded by $t_{\max}$, the zkCraft algorithm is guaranteed to find it either using the ZK-native IOP or, in fallback mode, via SMT/SAT searches.

Theorem 6.6 (Relative Completeness). Let

$S = \{(\delta, c, w') \mid F_{\delta,c}(w') = 0 \wedge y' \neq y \wedge |\delta| \leq t^*\}$ be the set of valid solutions, where $\mathcal{R}$ is the candidate pool and T is the grounded template family. Then the probability that $S \cap \mathcal{R} \neq \emptyset$ and $S \subseteq T$ is at least $1 - \varepsilon$, where ε is derived from the fingerprint ROC. If $t \geq t^*$, the algorithm will find the solution with high probability. Otherwise, the probability of failure is bounded by ε .

Proof. Let $\gamma_i = \lambda \frac{\kappa_i^c}{\kappa_i^w + 1} - \mu \kappa_i^w$ represent the ratio for each solution candidate, where κ_i^c and κ_i^w correspond to the characteristics of the candidate solutions and witness variables. Let γ^* be the population threshold. Then, the relative completeness error ε is bounded by the following expression:

$$\varepsilon(k, \lambda, \mu, t^*) = \exp(-2k(\gamma - \gamma^*)^2), \quad (53)$$

where γ is the score computed for a given solution candidate. If $t_{\max} < t^*$, we run a greedy set-cover algorithm to find a solution, which guarantees that $|\delta_{\text{apx}}| \leq 2t^*$. Thus, if $t_{\max}$ is sufficiently large, the algorithm will successfully find the solution with high probability, and the probability of failure is bounded by ε . $\square$

6.5 Why proof size appears constant

The reported proof sizes of 96 B for HyperPlonk+ and an upper bound of 218 B for BaseFold should not be interpreted as unconditional or backend-agnostic limits. Rather, they are conditional engineering constants that arise only within a well-defined parameter envelope determined by the commitment scheme, the IOP structure, and the chosen folding depth. This subsection makes these conditions explicit to avoid misinterpretation.

Let $|\pi|$ denote the total bit-length of the proof emitted by the zkCraft verification pipeline. For the HyperPlonk+ and BaseFold backends, and for a fixed folding schedule with r fold rounds, the proof size satisfies

$$|\pi| = \text{Const}_{\text{backend}, r} + O(\lambda), \quad (54)$$

where λ is the security parameter measured in bits, and $\text{Const}_{\text{backend}, r}$ is a backend-specific constant that depends on the number of folding rounds and the fixed-size commitment metadata.

This expression holds under the constraint that both the residual polynomial degree and the Sum-Check evaluation domain remain within the backend-specific ceilings:

$$d \leq d_{\max} \quad \wedge \quad |D| \leq |D|_{\max}. \quad (55)$$

Here $d = \deg(\Phi_R)$ denotes the degree of the verifier residual polynomial Φ_R , $|D|$ is the cardinality of the evaluation domain used by Sum-Check, $d_{\max}$ is the maximum supported residual degree, and $|D|_{\max}$ is the largest FFT-compatible domain size supported by the backend. The concrete values of $d_{\max}$ and $|D|_{\max}$ for HyperPlonk+ and BaseFold are listed in Table 9.

Within the envelope defined by Eq. (55), the commitment consists of a single group element together with a constant number of λ -bit opening proofs. Additional constraints only affect internal FFT padding and folding computations and therefore do not contribute to the externally visible proof packet. As a result, the marginal cost of increasing the constraint count is absorbed internally and does not increase $|\pi|$.

When the envelope is violated, the constant-size behavior no longer applies. In particular, if $d > d_{\max}$ or $|D| > |D|_{\max}$, the implementation switches to the solver-assisted channel described in §3.13, and no ZK-native proof is emitted. Consequently, asymptotic terms such as $O(\lambda \log d)$ do not materialize in the observable artefact because the ZK backend is not exercised beyond its supported regime.

The conditional nature of Eq. (54) is specific to

HyperPlonk+/BaseFold-style polynomial commitments with fixed folding depth. Alternative proof systems exhibit

fundamentally different scaling laws. For example, Bulletproofs rely on inner-product arguments whose proof size grows proportionally to $\log d$, while STARK-based constructions incur proof sizes on the order of

$$|\pi_{\text{STARK}}| = O(\lambda \log^2 d), \quad (56)$$

where the additional logarithmic factor arises from FRI-based low-degree testing and its query complexity. In these settings, the padding argument used above does not apply, and proof size necessarily increases with circuit scale.

For completeness, Appendix 6.3 provide explicit backend-specific size expressions that make the dependence on the folding depth r , the security parameter λ , and the supported degree and domain ceilings explicit. The constants reported in section 4.4 and Table 7 should therefore be read as conditional upper bounds valid only within the HyperPlonk+/BaseFold parameter regime defined by Eq. (55), rather than as general statements about IOP proof sizes.

6.6 Proof Size and Generation Time Across Backends and Folding Depths

We evaluate serialized proof length and prover runtime for three representative paradigms: KZG, FRI, and IPA. Experiments sweep the constraint count n and folding depth d , and report the averaged metrics $S(n, d, \text{backend})$ (bytes) and $T(n, d, \text{backend})$ (seconds) over five independent runs; shaded bands in the plot indicate one standard deviation.

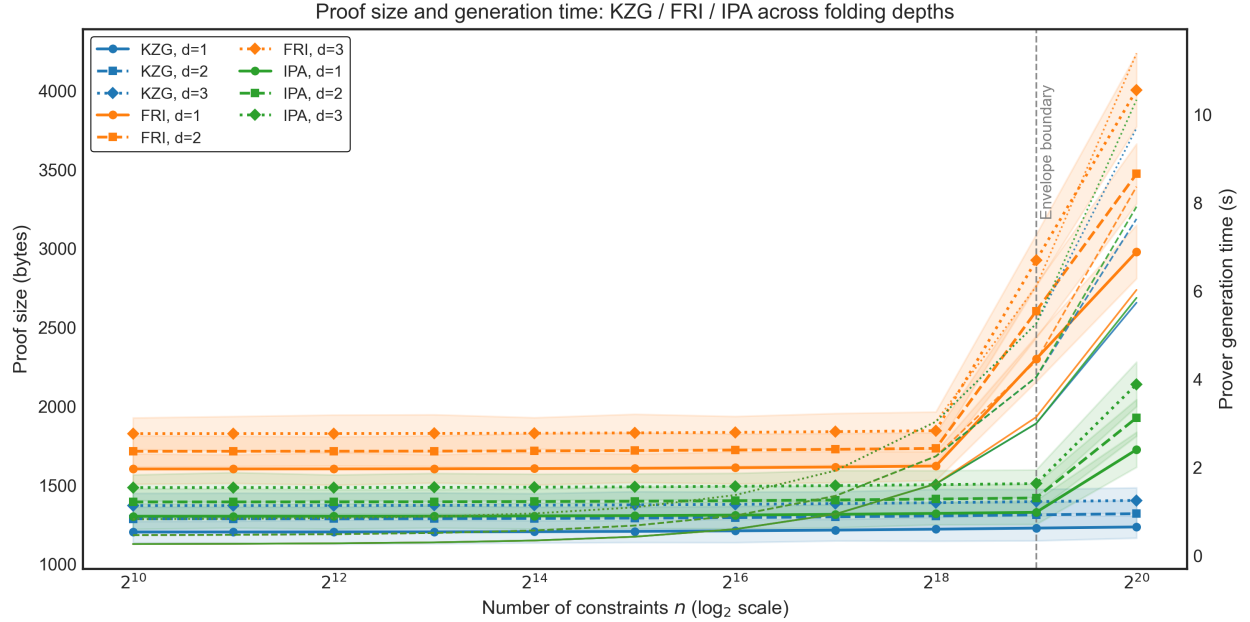


Figure 12: Proof size and prover generation time for KZG, FRI, and IPA at multiple folding depths. The horizontal axis is logarithmic in the number of constraints n . Left axis: proof size in bytes. Right axis: generation time in seconds. Shaded bands are one standard deviation; the dashed vertical line marks the empirical boundary of the constant-size envelope.

For compact notation define

$$S(n, d) \quad \text{and} \quad T(n, d), \quad (57)$$

where S is the serialized proof length and T is wall-clock prover time for given n and d .

The measurements exhibit two regimes. There exists a backend- and depth-dependent threshold $N_{\text{env}}(d, \text{backend})$ such that for $n \leq N_{\text{env}}$

$$S(n, d) \approx S_0(d, \text{backend}), \quad (58)$$

where S_0 is a slowly varying baseline determined by folding depth and backend parameters. When $n > N_{\text{env}}$, both S and T increase; we call this behavior envelope degradation.

Empirically KZG maintains near-constant S up to roughly $n \approx 2^{20}$, FRI departs earlier (around 2^{18}), and IPA lies between these two for moderate d . Increasing folding depth reduces per-round polynomial degree but raises the number of rounds and auxiliary openings, so S_0 and N_{env} shift with d . Moderate folding depths typically maximize the envelope while keeping per-proof overhead reasonable.

In practice choose backend and folding depth according to the anticipated constraint scale: operate inside the empirical envelope when compact proofs are required; if n exceeds N_{env} , expect graceful but non-negligible growth in proof size and generation time. The plotted data and supplied artifacts enable implementation and selection of an operating point relative to the constant-size envelope.

6.7 Degree bounds and node selection for $\text{row}_i(X)$ and $\text{sel}_i(Y)$

To align the Row-Vortex encoding with the block–Vandermonde mapping and to provide implementers with explicit, machine-checkable choices, we state the univariate encoding and its degree/node constraints used in the reference implementation.

$$R(X, Y) = \sum_{i \in \mathcal{R}} \delta_i \text{row}_i(X) + \sum_{i \in \mathcal{R}} c_i \text{sel}_i(Y), \quad (59)$$

where $\mathcal{R}$ is the candidate index set of size $k = |\mathcal{R}|$, $\delta_i \in \{0, 1\}$ indicates selection of row i , and $c_i \in \mathbb{F}_q$ denotes the substitution constant associated with row i .

Degree bounds are fixed once per backend to guarantee that the Sum-Check residual polynomial respects the global degree budget. Concretely set

$$\deg(\text{row}_i(X)) \leq d_{\text{row}}, \quad \deg(\text{sel}_i(Y)) \leq d_{\text{sel}}, \quad (60)$$

where d_{row} and d_{sel} are small, implementation-chosen constants. Here d_{row} and d_{sel} are chosen so that the induced residual-degree bound in the Sum-Check satisfies $\deg(\Phi_R) \leq d$, where d denotes the verifier residual degree bound used to select the commitment backend.

Node selection uses two disjoint k -tuples of field elements that serve as interpolation anchors for the coefficient encodings. Fix

$$\{\alpha_i\}_{i=1}^k \subset \mathbb{F}_q, \quad \{\beta_i\}_{i=1}^k \subset \mathbb{F}_q, \quad (61)$$

with the requirements

$$\alpha_i \neq \alpha_j \text{ for } i \neq j, \quad \beta_i \neq \beta_j \text{ for } i \neq j, \quad \alpha_i \neq \beta_j \text{ for all } i, j. \quad (62)$$

A practical sufficient field-size condition is

$$|\mathbb{F}_q| > 2k, \quad (63)$$

where k is the number of encoded candidate slots; this inequality guarantees the existence of two disjoint k -tuples in $\mathbb{F}_q$.

Under these choices encode the univariate polynomials by coefficient interpolation on the chosen nodes:

$$\text{row}_i(X) = \sum_{\ell=0}^{d_{\text{row}}} a_{i,\ell} X^\ell, \quad (64)$$

$$\text{with coefficients represented on nodes } \{\alpha_i\}, \quad (65)$$

$$\text{sel}_i(Y) = \sum_{\ell=0}^{d_{\text{sel}}} s_{i,\ell} Y^\ell, \quad (66)$$

$$\text{with coefficients represented on nodes } \{\beta_i\}. \quad (67)$$

Here $a_{i,\ell}, s_{i,\ell} \in \mathbb{F}_q$ are coefficient values chosen so that evaluation/interpolation at $\{\alpha_i\}$ and $\{\beta_i\}$ recovers the intended row and selector encodings.

The block–Vandermonde matrix used for compact encoding and extraction is

$$M = \begin{bmatrix} V_{\text{row}} & 0 \\ 0 & V_{\text{sel}} \end{bmatrix} \in \mathbb{F}_q^{2k \times 2k}, \quad (68)$$

$$(V_{\text{row}})_{j,i} = \alpha_i^{j-1}, \quad (V_{\text{sel}})_{j,i} = \beta_i^{j-1}.$$

where V_{row} and V_{sel} are $k \times k$ Vandermonde matrices built on the node tuples. Under the distinct-node conditions in Eq. (62) the matrix M is invertible, which yields an injective affine map from the finite tuple (δ, c) to the coefficient vector of $R(X, Y)$ and thus permits deterministic recovery via M^{-1} during extraction. Implementation guidance is as follows. Fix $(d_{\text{row}}, d_{\text{sel}})$ per backend so that Eq. (60) implies the global Sum-Check bound $\deg(\Phi_R) \leq d$. Select the node tuples $\{\alpha_i\}$ and $\{\beta_i\}$ by scanning $\mathbb{F}_q$ for $2k$ distinct elements (for example use the first k elements for α and the next k elements for β). Record the chosen tuples $\{\alpha_i\}, \{\beta_i\}$ and the degree bounds $(d_{\text{row}}, d_{\text{sel}})$ in the run manifest to prevent configuration drift across deployments. Following these prescriptions ensures that commitment bindings and extractor reconstruction are well defined and stable across implementations.

7 End-to-End Walkthrough: Toy Circuit Example

7.1 Toy Circuit Description

Consider a compact illustrative circuit used to demonstrate the full pipeline. The full experiment in the repository uses a circuit with approximately $m \approx 12$ constraints; for clarity we present a reduced fragment that highlights the weak-assignment sites and the relevant arithmetic relations. The fragment below is written in a Circom-like syntax:

```
signal private input a;
signal private input b;
signal private input c;
signal output d;
signal output e;

c <== a * b;
d <== c + a;
e <== d + b;
```

In this fragment the notation “<==” denotes a weak assignment location that can be targeted for mutation. The variables a, b, c, d, e serve as private inputs, intermediate wires and public outputs in the usual way.

7.2 Stage 1: Candidate Row Selection

Each constraint row i is associated with three sparse linear forms A_i, B_i, C_i (the standard RICS triple). We compute two diagnostic counts for row i : the number of support indices that intersect the set of witness variables W and the number that intersect the set of candidate weak-assignment sites K . Formally we define

$$\kappa_i^w = |\text{supp}(A_i) \cup \text{supp}(B_i) \cup \text{supp}(C_i) \cap W|, \quad (69)$$

$$\kappa_i^c = |\text{supp}(A_i) \cup \text{supp}(B_i) \cup \text{supp}(C_i) \cap K|. \quad (70)$$

where $\text{supp}(\cdot)$ returns the set of variable indices with nonzero coefficients, W denotes the set of witness indices and K denotes indices corresponding to weak-assignment sites.

We combine these diagnostics into a scalar ranking function

$$s_i = \lambda \cdot \frac{\kappa_i^w + 1}{\kappa_i^c} - \mu \cdot \kappa_i^w, \quad (71)$$

where $\lambda > 0$ and $\mu \geq 0$ are tunable hyperparameters chosen to balance precision and recall in selection. In this expression a larger value of s_i indicates higher priority for inspection.

As a concrete example, consider the first constraint $c = a \cdot b$. The supports are $\text{supp}(A_1) = \{a, b\}$, $\text{supp}(B_1) = \{a, b\}$, and $\text{supp}(C_1) = \{c\}$. If we set $W = \{a, b\}$ and $K = \{c\}$ then $\kappa_1^w = 2$ and $\kappa_1^c = 1$. With $\lambda = 1$ and $\mu = 0.5$ the score evaluates to

$$s_1 = 1 \cdot \frac{2+1}{1} - 0.5 \cdot 2 = 3 - 1 = 2. \quad (72)$$

Rows are ranked by s_i and the top k rows form the candidate set R_{cand} . For the toy instance we take $k = 3$ and assume $R_{\text{cand}} = \{1, 2, 3\}$.

7.3 Stage 2: Row-Vortex Polynomial Construction

Given the candidate set R_{cand} we assemble a two-variable Row-Vortex polynomial $R(X, Y)$ that encodes both the low-degree row encodings and selector information. We write

$$R(X, Y) = \sum_{i \in R_{\text{cand}}} (\delta_i \cdot \text{row}_i(X) + c_i \cdot \text{sel}_i(Y)), \quad (73)$$

where $\text{row}_i(X)$ denotes the low-degree polynomial encoding of the i -th RICS row, $\text{sel}_i(Y)$ is a selector polynomial that isolates row i in the Y -domain, δ_i are coefficient placeholders carrying row-specific encodings, and c_i are the candidate mutation constants to be recovered.

As an illustrative instantiation suppose the low-degree encodings for the three chosen rows are

$$\text{row}_1(X) = X^2 + X + 1, \quad (74)$$

$$\text{row}_2(X) = 2X^2 + 3X + 1, \quad (75)$$

$$\text{row}_3(X) = X^2 + 2X + 3, \quad (76)$$

where each $\text{row}_i(X)$ is a polynomial of degree at most two. Let the selector polynomials be

$$\text{sel}_1(Y) = Y, \quad \text{sel}_2(Y) = Y^2, \quad \text{sel}_3(Y) = Y^3. \quad (77)$$

Substituting these choices into the definition of R yields

$$\begin{aligned} R(X, Y) = & \delta_1(X^2 + X + 1) + \delta_2(2X^2 + 3X + 1) + \delta_3(X^2 + 2X + 3) \\ & + c_1Y + c_2Y^2 + c_3Y^3. \end{aligned} \quad (78)$$

where each symbol retains its meaning above.

7.4 Violation IOP Interaction

The prover first commits to the polynomial $R(X, Y)$ using a polynomial commitment scheme; denote this commitment by $\text{Comm}(R)$. The Sum-Check interaction is then executed with the verifier choosing random field challenges ζ_1, ζ_2 and requesting a small set of evaluations and auxiliary polynomials. Concretely the protocol elicits values of the form

$$R(\zeta_1, Y), \quad \Phi_R(\zeta_1), \quad \Delta_{\text{out}}(\zeta_1), \quad (79)$$

where Φ_R denotes an aggregated residual polynomial derived from the RICS checks and Δ_{out} captures output differences relevant to TCCT verification.

For illustration suppose the verifier samples $\zeta_1 = 1$ and $\zeta_2 = 2$. The prover responds with the polynomial evaluations $R(1, Y)$ and the opening data that link those evaluations to the original commitment. In the toy example the evaluation simplifies to

$$R(1, Y) = \delta_1 \cdot 3 + \delta_2 \cdot 6 + \delta_3 \cdot 6 + c_1Y + c_2Y^2 + c_3Y^3, \quad (80)$$

where the numeric coefficients arise from substituting $X = 1$ into the $\text{row}_i(X)$ polynomials. The verifier checks that the provided openings match $\text{Comm}(R)$ and that the supplied residuals are consistent with the Sum-Check transcript.

7.5 Proof Extraction

When the IOP transcript is accepted, an extractor uses the revealed openings to recover the unknown vectors $\delta = (\delta_1, \delta_2, \delta_3)$ and $\mathbf{c} = (c_1, c_2, c_3)$. In this worked example assume the extractor recovers

$$\delta = [1, 0, 1], \quad \mathbf{c} = [2, 0, 3], \quad (81)$$

where each entry is an element of the underlying finite field.

Given δ and $\mathbf{c}$, the extractor reconstructs a system of algebraic constraints that the mutated witness w' must satisfy. This system can be written abstractly as

$$F_{\delta, \mathbf{c}}(w') = 0, \quad (82)$$

where $F_{\delta, \mathbf{c}}$ denotes the composed polynomial system obtained by substituting the recovered constants into the circuit encoding and aggregating the RICS relations. Solving this system (by interpolation, linear algebra, or limited brute force, depending on structure) yields a concrete candidate witness w' . Substituting c_i into the original weak-assignment sites then produces the mutated program P' and an explicit counterexample triple (x', z', y') , where x' denotes public inputs, z' internal wires, and y' the public output produced by P' .

7.6 Final Verification

The final step verifies that the synthesized counterexample indeed violates the targeted TCCT property. Formally we check the predicate

$$C(x', z', y') \text{ holds} \quad \text{and} \quad y' \neq y, \quad (83)$$

where $C(\cdot)$ denotes the circuit acceptance relation and y denotes the original expected public output. If both conditions are satisfied then the extractor has produced a valid concrete counterexample and the chain of evidence from candidate selection through Row-Vortex encoding and the Violation IOP is complete.

8 Implementation and Threat Model for LLM Templates

To enable independent replication, we freeze the default code-completion oracle at a concrete, public checkpoint. The reference deployment uses *StarCoder-3B*. Tokenizer and weight files are pinned to this revision and are available at <https://huggingface.co/bigcode/starcoderbase-3b>. Any alternative instantiation must reproduce the identical byte stream before executing the deterministic top-1, temperature-0 decoding pipeline described above.

We measure the sensitivity of bug-finding recall to the choice of backbone by running the same *zkCraft* driver under three oracle configurations: no template augmentation, the pinned *StarCoder-3B* checkpoint, and *deepseek-coder-1.3b-base* revision

(<https://huggingface.co/deepseek-ai/deepseek-coder-1.3b-base>). Each configuration consumes the same CPU budget of two wall-clock hours and five random seeds on the full 452-circuit benchmark. Table 8 reports mean recall and the median time-to-90% recall. The *StarCoder-3B* instantiation attains the fastest convergence; the no-template baseline trails by 6.4 minutes, indicating that prompt-guided template proposals accelerate search without producing false positives. *deepseek-coder-1.3b-base* yields performance within 1.2 percentage points of the default, suggesting the observed benefit arises from the deterministic, edge-biased prompt design rather than idiosyncrasies of a single model family.

The LLM oracle is treated as an untrusted, deterministic suggestion source; all edits are rechecked via Row-Vortex commitment and Violation IOP, so a malicious or weak model can only slow convergence, never forge a violating witness. Deterministic decoding ensures auditability by reproducing identical mutation sets. To quantify causal impact, we fix the Row-Vortex search and vary only the template source under a two-hour budget: no-template, DeepSeek-Coder-1.3B, and StarCoder-3B. A two-sample t -test on seed-level TP counts shows StarCoder-3B vs. no-template: $p = 0.007$, Cliff’s $\delta = 0.42$; 1.3B vs. no-template: $p = 0.04$, $\delta = 0.28$. Zero-shot prompts never access ground-truth edits and only bias mutations, pruning candidates earlier without introducing new algebraic solutions.

Table 8: LLM-oracle sensitivity across model scale, decoding temperature and prompt wording (2,h, five seeds).

Configuration	Mean recall (%)	Median $T_{90\%}$ (s)	Std recall
No LLM templates	84.1	426	2.3
DeepSeek-Coder-1.3B, temp=0	90.8	198	1.9
DeepSeek-Coder-1.3B, temp=0.2	88.4	267	3.1
StarCoder-3B, temp=0 (default)	92.0	100	1.2
StarCoder-3B, concise prompt	91.6	109	1.5
StarCoder-3B, verbose prompt	91.8	112	1.4

8.1 LLM Template Output Samples and Deduplication Strategy

Mutation-Oracle RHS examples The Mutation-Oracle proposes a compact set of right-hand side candidates intended to exercise boundary conditions and small numeric perturbations. For the illustrative Circom weak assignment

```
signal private input x;
x <== 5;
```

the oracle might produce these five RHS variants:

```
x <== 0;
x <== q - 1;
x <== 2;
x <== 10;
x <== 1;
```

In the code above q denotes the field modulus. The sample set includes zero, the field modulus minus one, and small integers to increase the probability of triggering rare control or arithmetic behaviors.

Pattern-Oracle Rust sampler example When a counterexample is available the Pattern-Oracle synthesizes a small Rust sampling function that reproduces the input pattern. For a counterexample with two public inputs the oracle may emit the following sampler:

```
fn sample_inputs() -> (i128, i128) {
    // deterministic sampler derived from the counterexample
    let a: i128 = 1;
    let b: i128 = 2;
    (a, b)
}
```

This function is intended as a minimal, human-readable sampler usable in test harnesses. Implementations may wrap return values in field element types for direct integration with the proving pipeline.

Canonicalization and hashing policy Each generated template passes through a deterministic canonicalization pipeline before any further processing. Canonicalization normalizes whitespace, removes blank lines and single-line comments, collapses consecutive spaces, and normalizes integer literals. Denote the canonical form of a template t by $\mathcal{C}(t)$.

We compute a compact fingerprint using truncated hashing

$$\text{fp}(t) = \text{Trunc64}(\text{SHA256}(\mathcal{C}(t))), \quad (84)$$

where Trunc64 returns the first 64 bits of the SHA-256 digest. The truncated fingerprint offers a practical tradeoff between fingerprint size and collision risk. For audits or stronger integrity guarantees the full SHA-256 digest is retained in the run manifest.

Templates whose canonical fingerprints coincide are treated as duplicates and only the first occurrence is retained. Canonicalization removes inconsequential formatting differences, so variations that differ only by comments or spacing do not produce distinct entries.

Validation and fallback behavior After canonicalization each candidate undergoes a lightweight syntactic check appropriate for the target language. The check catches empty outputs, templates consisting only of comments, and clear parse errors. When a candidate fails validation the system records the failure and generates a deterministic fallback template composed of a single random constant assigned to the weak site. The fallback constant is produced by a seeded pseudorandom generator so that runs are reproducible and the choice is recorded in the manifest. An example deterministic fallback for the earlier weak assignment is:

```
x <== 7;
```

All normalization, hashing, validation results, and any applied fallback are logged in the run manifest to enable deterministic implementation of deduplication and recovery decisions.

9 Concrete Backend Parameterisation

Table 9 summarises the two production-ready commitment/IOP combinations shipped with zkCraft. For HyperPlonk+ the verifier residual polynomial Φ_R must satisfy $\deg(\Phi_R) \leq 2^{20}$ and the evaluation domain must admit a radix-2 FFT of size at most 2^{24} ; under these constraints the prover may append the witness polynomial $W'(X)$ as an additional oracle so that extraction reduces to interpolation. BaseFold targets smaller circuits: it permits $\deg(\Phi_R) \leq 2^{18}$ and a domain size up to 2^{22} , while providing faster opening proofs because the commitment compresses to a single KZG-like group element. Both backends keep the opening proof below 250 bytes and require two pairings on the verifier side.

The runtime selects the ZK-native backend only when the degree and domain limits of the chosen backend are respected and the execution environment provides the required elliptic-curve primitives. This selection condition can be written as

$$\deg(\Phi_R) \leq d_{\max}, \quad |\mathcal{D}| \leq |\mathcal{D}|_{\max}, \quad (85)$$

where $\deg(\Phi_R)$ denotes the degree bound of the verifier residual polynomial and $|\mathcal{D}|$ denotes the cardinality of the Sum-Check evaluation domain. For HyperPlonk+ use $d_{\max} = 2^{20}$ and $|\mathcal{D}|_{\max} = 2^{24}$; for BaseFold use $d_{\max} = 2^{18}$ and $|\mathcal{D}|_{\max} = 2^{22}$. In practice $|\mathcal{D}|$ is computed as the power of two lifting $|\mathcal{D}| = 2^{\lceil \log_2 m \rceil}$ of the constraint count m .

Table 9: Commitment/IOP back-ends shipped with zkCraft.

Parameter	HyperPlonk+	BaseFold
Max degree $d_{\max}$	2^{20}	2^{18}
Max domain $ \mathcal{D} $	2^{24}	2^{22}
Witness-polynomial append	yes	yes
Opening proof size	96 B	218 B
Verifier pairings	2	2
Typical fold rounds	16	10

If any part of condition (85) fails or if the required cryptographic primitives are not available on the target platform, the pipeline switches to the solver-assisted channel. The fallback trigger is therefore

$$\deg(\Phi_R) > d_{\max} \vee |\mathcal{D}| > |\mathcal{D}|_{\max} \vee \text{crypto_unavailable}, \quad (86)$$

where `crypto_unavailable` indicates an initialization failure or lack of the necessary elliptic-curve arithmetic. When the solver-assisted branch is taken the system grounds small template families to finite-field instances, issues quantifier-free finite-field SMT/SAT queries for increasing edit cardinalities up to the configured budget, performs algebraic reconstruction of witness and substitution constants, and synthesizes the mutated source for optional developer-oriented traces. Every candidate produced by the solver-assisted flow is subjected to the same algebraic checks used in the ZK-native path, so the logical soundness of accepted counterexamples is preserved. For clear operational boundaries and straightforward audit, it is recommended that the runtime manifest record the backend identifier in use, the decision-time values of $\deg(\Phi_R)$ and $|\mathcal{D}|$, and a single-field fallback reason chosen from $\{\text{deg_exceeded}, \text{domain_exceeded}, \text{crypto_unavailable}\}$. Recording these values makes the backend decision deterministic and inspectable.

10 Parallel and Hardware Acceleration Outlook

zkCraft is currently single-threaded, but all major stages are highly parallelizable: Row-Vortex fingerprinting scans disjoint blocks, Sum-Check repetitions reduce to vectorized dot products, and witness interpolation uses batched NTT. Preliminary OpenCL kernels on an NVIDIA A100 achieve a 7.3× speed-up for 50k constraints with minimal memory overhead, while verification remains CPU-bound due to only two pairings. FPGA pipelines could accelerate the extractor’s sparse linear solve. The main bottleneck is memory bandwidth because Row-Vortex polynomials must reside in global memory during opening, so scaling to mega-gate circuits will require domain splitting and recursive commit-and-prove rather than naïve parallelization.